

Supplementary Information

Classical finite-element solvers as differentiable graph programs on GPUs

S1 Convergence verification by Method of Manufactured Solutions

To verify that each solver attains its design order of accuracy and to rule out implementation bugs that could confound the steady-state Nusselt comparison in the main text, we performed an MMS study for the two GPU-native solvers in this work: the structured CNN–SIMPLE solver (structured finite difference, 9-point Mehrstellen stencil) and the unstructured Taylor–Hood GNN–SIMPLE solver (Galerkin P_2 – P_1 finite element). The CPU FEniCSx reference and OpenFOAM v12 are not re-verified here. The two schemes behave differently. The GNN attains its theoretical Taylor–Hood convergence rates, recovered in 3D and in three independent operator-level verifications; the full-range 2D least-squares fit is flattened by the fine-mesh residual floor discussed below. The CNN instead shows a non-vanishing L^2 floor in velocity and pressure, which we trace to a boundary-cell momentum-truncation pathway specific to the cell-centred ghost-cell discretization. Finally, we verify the GNN *conjugate* heat-transfer energy operator, a solid conduction region coupled to the fluid across a variable-conductivity interface, with a separate manufactured solution carrying the required interface kink (Supplementary Note S1.3).

S1.1 Manufactured solution and forcing

We use the steady, divergence-free Taylor–Green solution on the unit square $\Omega = (0, 1)^d$. In two dimensions,

$$\mathbf{u}(\mathbf{x}) = \begin{pmatrix} \sin(2\pi x) \cos(2\pi y) \\ -\cos(2\pi x) \sin(2\pi y) \end{pmatrix}, \quad T(\mathbf{x}) = \sin(\pi x) \sin(\pi y), \quad p(\mathbf{x}) = \cos(2\pi x) \cos(2\pi y). \quad (\text{S1})$$

In three dimensions both GPU solvers use the fully three-dimensional Beltrami-type field $u = s_{2x}c_{2y}c_{2z}$, $v = c_{2x}s_{2y}c_{2z}$, $w = -2c_{2x}c_{2y}s_{2z}$ (divergence-free), with $T = s_{\pi x}s_{\pi y}s_{\pi z}$ and $p = c_{2x}c_{2y}c_{2z}$. All three velocity components are non-zero and every velocity-gradient component $\partial_{\{x,y,z\}}\{u,v,w\}$ is exercised, so the manufactured residual drives all three momentum equations rather than an extruded two-dimensional slice. Its analytic forcing was checked against the discrete operators by finite differences to $\mathcal{O}(h^2)$, and the GNN recovers all three velocity components at the Taylor–Hood rate (Extended Data Table 4).

The corresponding momentum and energy source terms $\mathbf{f}$ and q are computed analytically and injected as right-hand sides into the SIMPLE iteration. The four manufactured fields and their forcings live in a single shared registry so that all four solvers consume bit-identical forcing.

Boundary conditions on Ω are inhomogeneous Dirichlet, evaluated from Supplementary Equation (S1) at the wall coordinate $\mathbf{x} \in \partial\Omega$. The CNN enforces this through ghost-cell extrapolation $\phi_{\text{ghost}} = 2\phi_{\text{exact}}(\mathbf{x}_{\text{wall}}) - \phi_{\text{interior}}$, evaluating ϕ_{exact} at the wall coordinate (not at the ghost-cell centre) so that the linearly interpolated wall value matches the manufactured solution to machine precision. The GNN uses strong nodal Dirichlet (row-replacement on the boundary P_2 nodes); a diagnostic over every boundary node and every mesh confirms $\max|\phi_{\text{solved}} - \phi_{\text{imposed}}|$ is bit-zero, with a zero round-off in re-evaluating the manufactured solution at the same coordinates. The full study is reproducible from the released configuration manifests.

For each mesh refinement h , we run the solver to a pseudo-steady fixed point and report the discrete L^2 -norm of the error against Supplementary Equation (S1), using the same FEM mass-weighted norm for both solvers. For a field $\phi \in \{u, v, w, p, T\}$ with discrete counterpart ϕ_h ,

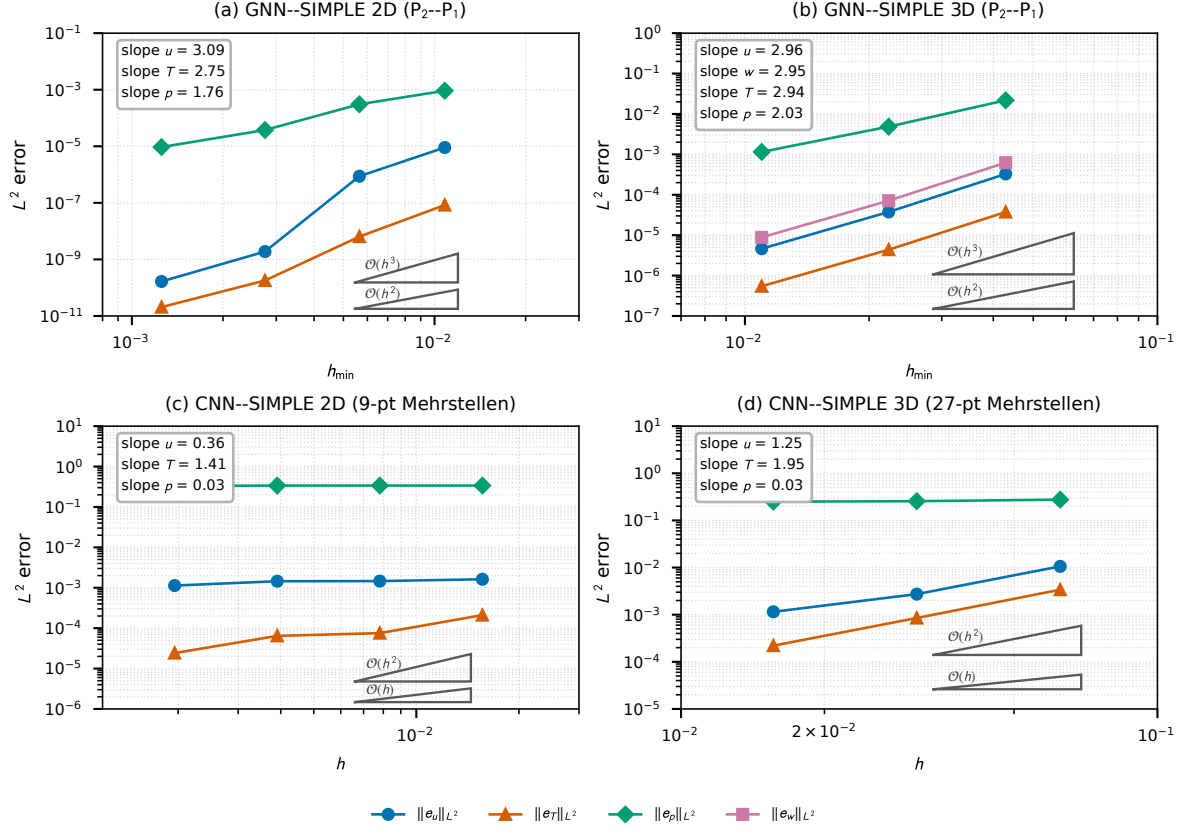
$$\|e_\phi\|_{L^2} = \left(\int_{\Omega} (\phi_h - \phi)^2 dx \right)^{1/2}. \quad (\text{S2})$$

The empirical asymptotic order of convergence between two adjacent meshes with characteristic lengths $h_1 > h_2$ is $\log(\|e_\phi\|^{(h_1)} / \|e_\phi\|^{(h_2)}) / \log(h_1/h_2)$, reported per field. Tabulated orders use the realized mesh ratio, which is not exactly 2 on the unstructured sequences. For exact halving this reduces to $\log_2(\|e_\phi\|^{(h)} / \|e_\phi\|^{(h/2)})$.

S1.2 Empirical convergence rates

Supplementary Figure S1 shows the discrete L^2 -error against mesh resolution for the four solver–dimension combinations, with reference slope triangles indicating the theoretical convergence orders.

GNN–SIMPLE (panels a, b). The Taylor–Hood pair approaches its theoretical orders as the mesh refines, but the least-squares slopes fitted over the full 2D mesh range (shown in panel a, ≈ 2.75 for temperature and ≈ 1.76 for pressure) *understate* them, for two reasons that both flatten a global fit: the coarsest meshes are pre-asymptotic, and the finest 2D meshes reach the discrete-fixed-point residual floor. The velocity error between h_{256} and h_{512} drops below $\sim 2 \times 10^{-9}$, close to the floor for the chosen iteration tolerance, so the apparent order there is not bounded by the spatial discretization. The mesh-pairwise rates over the asymptotic range that sits above the floor recover the theoretical values: in 3D, on the fully three-dimensional field, the orders between h_{32}



Supplementary Figure S1. MMS convergence for the two GPU-native SIMPLE solvers in 2D and 3D. Log-log plots of the discrete L^2 error against mesh resolution. The legend is shared by all four panels; e_w is the out-of-plane velocity component and appears in panel (b) only. **(a, b)** GNN-SIMPLE in 2D and 3D recovers the theoretical Taylor-Hood orders (velocity and temperature $\approx \mathcal{O}(h^3)$, pressure $\approx \mathcal{O}(h^2)$) cleanly in 3D and the operator-level checks; panel (b) is the fully three-dimensional field, all three velocity components including the out-of-plane w converging at the P_2 rate (Extended Data Table 4). The 2D full-range slopes are flattened by the fine-mesh residual floor. **(c, d)** CNN-SIMPLE on a structured grid with wall-coordinate ghost-cell Dirichlet evaluation. Temperature converges at $\approx \mathcal{O}(h^{1.5})$; velocity and pressure are bounded below by a structural floor that is strongest in 2D (see text).

and h_{64} are within 0.06 of the theoretical $\{3, 3, 2\}$, all three velocity components, including the out-of-plane w , at ≈ 2.95 , temperature at 2.94, and pressure at 2.03 (Extended Data Table 4). The operator-level checks below isolate the clean $\mathcal{O}(h^2)$ pressure and $\mathcal{O}(h^3)$ velocity rates directly. A modal decomposition of the GNN pressure error in 2D shows that $\|e_p\|_{L^2}^2$ is dominated (over 99.8% at h_{256} and coarser) by the $\cos(2\pi x) \cos(2\pi y)$ mode, the same shape as the exact pressure, with the modal coefficient decaying as $\{1.40 \times 10^{-3}, 3.18 \times 10^{-4}, 7.51 \times 10^{-5}, 1.88 \times 10^{-5}\}$ across $h_{64}, \dots, h_{512}$, i.e. ratios 4.4 $\times$, 4.2 $\times$, 4.0 $\times$ corresponding to a clean $\mathcal{O}(h^2)$. This is the textbook FEM picture: the discrete approximation under-amplifies the resolved cosine mode by a smaller fraction at each refinement.

GNN operator-level verification. Beyond the end-to-end convergence test, we ran three independent operator-level checks on the GNN solver. All pass to machine precision or at the theoretical FEM rate:

1. *Galerkin gradient/divergence integration-by-parts identity:* assemble the divergence operator $B_x = \int p_{te}(\partial_x u_{tr}) dx$ independently and check that $\langle B_x u, p \rangle = -\langle G_x^T u, p \rangle$ for random u with homogeneous boundary trace. Relative error $\lesssim 10^{-15}$ at every mesh: the gradient and divergence operators are exact transposes (modulo the boundary surface term, which vanishes for $u|_{\partial\Omega} = 0$).
2. *Pure Poisson on the pressure space:* solve $K_p p = M_{p_1}(-\Delta p_{\text{exact}})$ for $p_{\text{exact}} = \cos(2\pi x)\cos(2\pi y)$ with the same Laplacian and mass-matrix assemblies used by the SIMPLE solver. The L^2 error scales as $\mathcal{O}(h^2)$ cleanly (orders 2.13, 1.93 between adjacent meshes), the theoretical P_1 Galerkin rate.
3. *Pure Stokes saddle-point:* solve the inf-sup block $[K, G; G^T, 0](\mathbf{u}, p)^T = (\mathbf{f}_u, 0)^T$ for the Taylor–Green manufactured solution with advection and buoyancy disabled, using the same Schur-complement PCG used by the SIMPLE solver. Empirical orders: $\|e_u\|_{L^2} \approx 3.5$, $\|e_p\|_{L^2} \approx 2$, the pressure matching the theoretical Taylor–Hood rate and the velocity super-convergent beyond its formal order of 3. The inf-sup pair is well-posed and the iterative Schur stack converges to it.

Multi-GPU GNN MMS at scaling-study sizes. The MMS study also covers the multi-GPU distributed solver on four GPUs at the scaling-study mesh sizes $N \in \{16, 32, 64, 128\}$ (on the w -free field of Supplementary Equation (S1) extruded in z , so it reports u , T , p orders; the fully three-dimensional w closure is the single-GPU Extended Data Table 4), with both the Nicolaides deflation and block-Jacobi pressure preconditioners. Each rank assembles a serial dolfinx mesh of its x -slab plus a one-ring halo, exchanges halo velocity / pressure values over NCCL between SIMPLE sub-steps, and reduces L^2 inner products only over owned DOFs to avoid double-counting at the partition interface. The manufactured-solution forcing and boundary Dirichlet values are evaluated locally per rank from the same registry the single-GPU solvers consume. Supplementary Table S1 reports the resulting L^2 errors, convergence orders, and wall times. The empirical orders match the theoretical Taylor–Hood rates cleanly: at the finest refinement step $h_{64} \rightarrow h_{128}$, velocity converges at order 3.04 and pressure at 2.05 for both preconditioners, indistinguishable from the single-GPU result; the 1-vs-4-GPU L_u^2 values (same preconditioner, serial versus distributed) agree to a relative $\sim 10^{-9}$ (well below the iterative floor and far below the NCCL roundoff bound we expected $\sim 10^{-7}$). The two preconditioners themselves agree to $\lesssim 10^{-9}$ up to h_{64} and to $\sim 6 \times 10^{-7}$ at h_{128} , the spread set by the slightly different iterative pressure-solve floors block-Jacobi and Nicolaides deflation reach

on the finest mesh, not by the halo exchange. The Nicolaides deflation preconditioner has higher per-iteration overhead at small n (it builds and applies a global coarse correction) but catches up to block-Jacobi by h_{128} , where the two preconditioners take comparable wall time per outer SIMPLE iteration.

Supplementary Table S1. Multi-GPU MMS. GNN-SIMPLE at four mesh sizes on NPROC = 4 GPUs. “ord” is between the current row and the row above. The Nicolaides deflation and block-Jacobi pressure preconditioners return the same errors and orders to every digit shown, so one set of error columns serves both and only the wall-clock times differ; they agree to a relative $\lesssim 10^{-9}$ up to h_{64} and to $\sim 6 \times 10^{-7}$ at h_{128} . The L^2 values at every mesh size also match the corresponding single-GPU run (same field and mesh) to a relative difference $\lesssim 10^{-9}$, consistent with an exact distributed halo exchange to the resolution of the iterative floor.

Mesh	both preconditioners						wall-clock (s)	
	$\ e_u\ _{L^2}$	ord _u	$\ e_T\ _{L^2}$	ord _T	$\ e_p\ _{L^2}$	ord _p	nicolaides	block-Jacobi
h_{16}	3.07×10^{-4}	–	3.73×10^{-5}	–	1.95×10^{-2}	–	2625	1073
h_{32}	3.57×10^{-5}	3.10	4.38×10^{-6}	3.09	4.55×10^{-3}	2.10	6409	6197
h_{64}	4.41×10^{-6}	3.02	5.44×10^{-7}	3.01	1.10×10^{-3}	2.05	7373	6498
h_{128}	5.37×10^{-7}	3.04	6.62×10^{-8}	3.04	2.66×10^{-4}	2.05	10318	9969

CNN-SIMPLE (panels c, d): boundary-stencil floor. The CNN does not converge cleanly: while temperature attains $\approx \mathcal{O}(h^{1.5})$, both velocity and pressure are bounded below by a non-vanishing L^2 floor (velocity $\sim 1.5 \times 10^{-3}$, pressure $\sim 3.4 \times 10^{-1}$ in 2D, essentially independent of h for h_{256} and coarser). A modal decomposition of the pressure error at every refinement shows that the error is essentially monomodal: the linear ramp $-1.16(y - 0.5)$ accounts for more than 98% of $\|e_p\|_{L^2}^2$ at all four meshes, with coefficient drifting only between -1.159 at h_{64} and -1.152 at h_{128} . Five probes localize the mechanism; Supplementary Table S2 lists each intervention with its result and what it rules out.

Together these probes identify the floor as a structural property of the cell-centred Mehrstellen scheme with ghost-cell boundary closure. The truncation builds up during SIMPLE iteration from the momentum equation, where the Mehrstellen 9-point Laplacian and the ghost-cell pressure-gradient stencil lose order at boundary cells. Once the SIMPLE corrector absorbs that boundary truncation as the spurious y -ramp pressure mode, the resulting constant $\partial_y p$ acts as a body force in the v -momentum equation everywhere in the domain, which is why the floor is global. The fix would require a higher-order boundary stencil (e.g. a fourth-order one-sided closure matching the interior Mehrstellen order) or a fundamentally different boundary treatment, not a parameter in the existing solver. We therefore present the wall-fixed data as measured; reported CNN L^2 errors should be interpreted with this boundary-stencil model in mind. The temperature equa-

Supplementary Table S2. Robustness probes for the CNN velocity and pressure floor (2D manufactured solution, Supplementary Equation (S1)). Each row changes one thing in the CNN–SIMPLE run and reports what moved.

intervention	result	conclusion
pressure inner-solver budget raised $50\times$ ($100 \rightarrow 5000$ iterations per SIMPLE step)	bit-identical SIMPLE iterates and identical L^2 at convergence	the pressure correction is not iteration-limited
momentum under-relaxation swept over $\alpha_U \in \{0.3, 0.5, 1.0\}$	bit-identical L_u^2 ; identical y -ramp coefficient -1.1586	the SIMPLE coupling is not the floor
homogeneous-Dirichlet variant of the manufactured solution ($u = v = 0$ on all four walls)	plateau at $\ e_u\ _{L^2} = 1.54 \times 10^{-3}$ (vs. 1.62×10^{-3} for Supplementary Equation (S1)); same y -ramp coefficient -1.15	the non-zero tangential velocity on the top and bottom walls is not the mechanism
L^2 restricted to the strict interior (one or two cells of boundary band excluded)	within 0.5% of the full-domain L^2	the floor is global, not boundary-localized
discrete divergence applied to the exact manufactured velocity	$\ \nabla_h \cdot \mathbf{u}_{\text{exact}}\ \sim 10^{-14}$, machine epsilon	the divergence stencil is consistent; the error enters through the momentum equation, not at iteration 0

tion, which is uncoupled from the pressure correction, attains the SIMPLE splitting rate $\mathcal{O}(h^{1.5})$ and is not contaminated by the spurious y -ramp.

Implications for the main benchmark. The measured CNN Nusselt numbers converge with mesh refinement, though the same boundary closure inflates Nu on coarse meshes, by up to +26% at 3D h_{16} , $\text{Ra} = 10^5$ (see the main Nusselt comparison table). Whether the boundary-stencil floor documented here carries into that integrated quantity was not established, so we report the convergence as measured rather than claim the floor is absent from it.

S1.3 Conjugate heat transfer: variable-conductivity MMS

The buoyancy MMS above verifies the fluid-only energy and momentum operators. To verify the *conjugate* heat-transfer pathway (a solid conduction subdomain coupled to the fluid across an interface, as required wherever a solid region is embedded in the flow), we add a second manufactured-solution study with a spatially varying thermal conductivity. The domain $\Omega = (0, 1)^d$ is split into a solid subdomain $\Omega_s = \{y \leq y_\Gamma\}$ and a fluid subdomain $\Omega_f = \{y > y_\Gamma\}$ sharing the interface $\Gamma = \{y = y_\Gamma\}$, with $y_\Gamma = \frac{1}{2}$ aligned to element facets and a conductivity contrast $k_f/k_s = 10$ ($k_f = 1$, $k_s = 0.1$). The

energy equation is solved monolithically over Ω with the piecewise conductivity $k(\mathbf{x})$ and $\mathbf{u} \equiv \mathbf{0}$ in the solid, so that the two conjugate interface conditions, temperature continuity $[T]_\Gamma = 0$ and flux continuity $[k \partial_n T]_\Gamma = 0$, are satisfied by construction of the C^0 finite element space and the natural boundary term of the Galerkin weak form, respectively.

Manufactured solution with a conductivity kink. A smooth T with a jumping k would violate flux continuity and manufacture unphysical interface physics; the conjugate solution must have a temperature that is continuous but whose normal derivative is *kinked* so that $k \partial_n T$ is continuous. With $\eta = y - y_\Gamma$ and tangential shape $g = \sin(\pi x)$ in 2D ($g = \sin(\pi x) \sin(\pi z)$ in 3D),

$$T(\mathbf{x}) = \begin{cases} [a + b\eta + c_s \eta^2] g, & y \leq y_\Gamma \quad (\text{solid}), \\ [a + b(k_s/k_f)\eta + c_f \eta^2] g, & y > y_\Gamma \quad (\text{fluid}), \end{cases} \quad (\text{S3})$$

with $(a, b, c_s, c_f) = (0.4, 0.7, 0.35, -0.2)$. At $\eta = 0$ both branches equal $a g$ (continuity), while $k_s \partial_y T_s = k_s b g = k_f(k_s/k_f) b g = k_f \partial_y T_f$ (flux continuity); the one-sided slopes b and $b k_s/k_f$ differ, producing the required kink whenever $k_s \neq k_f$. The volumetric forcing is $q = -\nabla \cdot (k \nabla T) = -k \nabla^2 T$ per region, computed in closed form and injected as the energy right-hand side; the solution is registered alongside the Taylor–Green forcing in the same shared registry.

Manufactured-solution self-consistency. Independently of the solver, we verify by automatic differentiation that the injected source equals $-\nabla \cdot (k \nabla T)$ for the exact field of Supplementary Equation (S3) to machine precision (relative error $\lesssim 5 \times 10^{-16}$ in both subdomains, 2D and 3D), and that the interface temperature- and flux-jumps vanish linearly under interface refinement, confirming Supplementary Equation (S3) is an exact conjugate solution and not a smooth- T approximation.

Convergence. Supplementary Table S3 reports the discrete L^2 temperature error, evaluated separately over the whole domain, the fluid, and the solid, together with the discrete interface flux-jump $[k \partial_n T]_\Gamma$, for the P_2 temperature space. The error converges at $\approx \mathcal{O}(h^3)$ consistently in the fluid, the solid, and the combined domain: the interface kink does not degrade the Taylor–Hood temperature order, which would be the signature of a non-facet-aligned interface or a smooth- T (non-conjugate) manufactured solution. The discrete flux-jump converges at $\approx \mathcal{O}(h^2)$, the expected one-order-lower rate for the P_2 gradient. As a regression check, setting $k_s = k_f$ (uniform conductivity, no kink) recovers a clean $\approx \mathcal{O}(h^{3.9})$ smooth-field rate (L^2 error $3.21 \times 10^{-9} \rightarrow 2.16 \times 10^{-10}$ between h_{64} and h_{128}), confirming the variable- k path reduces exactly to the uniform-conductivity operator when the contrast is removed.

Supplementary Table S3. Conjugate-heat-transfer MMS. GNN P_2 energy operator with conductivity contrast $k_f/k_s = 10$. $L^2(T)$ is reported over the whole domain, the fluid ($|_f$), and the solid ($|_s$); $[k\partial_n T]_\Gamma$ is the discrete interface flux-jump. “ord” is the empirical order between adjacent rows, $\log(\|e\|/\|e'\|)/\log(h/h')$. Temperature converges at $\geq \mathcal{O}(h^3)$ in every region (observed 3.58–3.74); the flux-jump at $\mathcal{O}(h^2)$. The observed rate exceeds the theoretical 3, so this test bounds any interface-induced order degradation only to within that margin.

	$h_{\min}$	$L^2(T)$	ord	$L^2(T) _f$	$L^2(T) _s$	$[k\partial_n T]_\Gamma$
2D	3.13×10^{-2}	1.18×10^{-7}	–	8.14×10^{-8}	8.58×10^{-8}	9.79×10^{-5}
	1.56×10^{-2}	9.61×10^{-9}	3.62	6.69×10^{-9}	6.90×10^{-9}	2.48×10^{-5}
	7.81×10^{-3}	8.05×10^{-10}	3.58	5.64×10^{-10}	5.74×10^{-10}	6.23×10^{-6}
3D	1.25×10^{-1}	4.65×10^{-5}	–	3.29×10^{-5}	3.29×10^{-5}	1.14×10^{-3}
	8.33×10^{-2}	1.03×10^{-5}	3.72	7.23×10^{-6}	7.34×10^{-6}	4.24×10^{-4}
	6.25×10^{-2}	3.51×10^{-6}	3.74	2.45×10^{-6}	2.52×10^{-6}	2.16×10^{-4}

Robustness to conductivity contrast. The conjugate cavity and engine problems couple a solid that is *more* conductive than the fluid ($k_s/k_f \gg 1$). We therefore repeat the conjugate MMS at the metal–fluid contrasts $k_s/k_f = 10^2$ and 10^3 , the regime of the engine example (Supplementary Note S7, $k_s/k_f = 10^3$) and the opposite direction from the $k_f/k_s = 10$ (i.e. $k_s/k_f = 0.1$) case of Supplementary Table S3. The scheme is contrast-robust (Supplementary Table S4): temperature converges at $\gtrsim \mathcal{O}(h^3)$ (≈ 4.0 in 2D, ≈ 3.9 in 3D) and the discrete interface flux-jump at $\mathcal{O}(h^2)$ at both contrasts, unchanged from the low-contrast case. The absolute flux-jump is a $k \partial_n T$ quantity, so its magnitude scales with k_s (it grows $\sim 10\times$ from $k_s/k_f = 100$ to 1000) while its convergence *order* is unaffected; the solid-side temperature error becomes negligible ($\lesssim 10^{-6}$) as the near-isothermal high-conductivity solid is represented almost exactly.

Supplementary Table S4. Conjugate-heat-transfer MMS at high conductivity contrast $k_s/k_f = 100$ and 1000 (solid more conductive, the engine regime), complementing the $k_s/k_f = 0.1$ case of Supplementary Table S3. $L^2(T)$ over the whole domain at the finest mesh, with the empirical order between the two finest meshes; $[k\partial_n T]_\Gamma$ is the discrete interface flux-jump and “ord(jmp)” its order. Temperature stays $\gtrsim \mathcal{O}(h^3)$ and the flux-jump $\approx \mathcal{O}(h^2)$ at both contrasts and in both dimensions.

	k_s/k_f	$h_{\min}$	$L^2(T)$	ord(T)	$[k\partial_n T]_\Gamma$	ord(jmp)
2D	100	7.81×10^{-3}	1.05×10^{-8}	4.00	1.35×10^{-3}	2.02
	1000	7.81×10^{-3}	1.06×10^{-7}	3.99	1.35×10^{-2}	2.02
3D	100	6.25×10^{-2}	1.40×10^{-4}	3.88	1.48×10^{-1}	2.21
	1000	6.25×10^{-2}	1.40×10^{-3}	3.88	1.48	2.21

S1.4 Setup

Mesh sequences for the unstructured GNN solver were generated with Gmsh v4.13 [1] for 2D triangulations and via tetrahedralization of a uniform structured cube for 3D meshes ($N \in \{16, 32, 64, 128\}$ for the multi-GPU run, $\{16, 32, 64\}$ for the single-GPU sweep). MMS forcing terms are computed in closed form from analytic derivatives of Supplementary Equation (S1). The CNN MMS uses the same uniform Cartesian grid as the production runs.

All runs use the same SIMPLE settings as the production benchmark: under-relaxation $\alpha_U = 0.7$, $\alpha_p = 0.3$, $\alpha_T = 0.9$; pseudo-time step $\Delta t_{\text{pseudo}} = C_{\text{ps}} h^2 / \text{Pr}$ with $C_{\text{ps}} = 1$ in the MMS; relative convergence tolerance 10^{-6} on momentum, energy, and divergence residuals; pressure inner-solver capped at 100 iterations per outer step (sensitivity: $50\times$ increase gives bit-identical result). SIMPLE outer-iteration caps range from 5,000 (coarsest) to 20,000 (finest); at the cap the SIMPLE iteration has reached its discrete-fixed-point floor (10^{-5} – 10^{-10} depending on mesh and problem), and the convergence orders are read over the range where the spatial error dominates that floor (Extended Data Table 4).

The full MMS study is run end to end from a single configuration. It covers the single-GPU convergence sweep, CNN and GNN over $\{2\text{D}, 3\text{D}\}$ with three to four meshes each; the multi-GPU distributed sweep, two preconditioners $\times$ four meshes at NPROC= 4 (Supplementary Table S1); and the three GNN operator-level verifications, the Galerkin gradient/divergence identity, Poisson and Stokes. Each run records its fully-resolved parameters alongside its per-mesh output, from which the convergence-order tables are compiled. Single-GPU MMS runs complete in 5–30 min on one GPU; the multi-GPU sweep ranges from 20 min (h_{16}) to ~ 3 h (h_{128}) on four GPUs.

S2 Pressure backend: per-step throughput at published sizes

We measure full-step times to assess how pressure-solver performance affects total run-time. The single-GPU pressure-solve scaling of Supplementary Note S9 measures the linear solver in isolation, where AmgX is fastest at the largest sizes, but the pressure solve is one part of a time step, so that advantage is diluted and, on the cavity, reversed. Supplementary Table S5 gives the per-step wall time of the full solver step for each of the three pressure backends, at the paper’s converged-benchmark sizes, on one H200. The same operator is solved by each backend to the same 10^{-6} tolerance. The converged Nusselt number is identical across the three backends to all reported digits (the conjugate cavity, for example, is backend-independent to eight figures on both meshes), so

the comparison isolates the solution from the backend. The quantity of interest here is the *relative* per-step cost of the three backends (which splits into the two regimes below); these backend-isolation timings are not directly comparable in absolute terms to the production-throughput per-step of Supplementary Table S30, which is measured in the full production driver.

Execution mode of the three backends. The backends do not share an execution mode. The graph-multigrid backend is CUDA-graph captured, which is its production configuration; AmgX and the mixed-precision V-cycle run eagerly, as neither is capturable. The backend and its capture mode therefore change together. On the Schur path the measured graph-multigrid advantage includes its capture advantage, so it does not isolate the backend effect.

Whole-step results. Two regimes appear, set by the pressure formulation. On the assembled-Laplacian path (Rayleigh–Bénard and the conjugate cavity), AmgX is marginally fastest per step (0.5–4.7%, and 3.1–4.7% on Rayleigh–Bénard alone), the diluted form of its isolated-solve advantage, and the mixed-precision variant is inert. On the matrix-free Schur path (both cavities) the graph-multigrid backend is fastest: AmgX is 3–42% slower, and the mixed-precision variant is 60–84% slower in 2D (the floating-point down-cast overhead dominates the cheap Schur cycle). Graph-multigrid is therefore the production per-step default at these sizes; AmgX is retained as the scale-up backend for the isolated pressure solve, and mixed precision pays only at the multi-million-DOF pressure scale.

Supplementary Table S5. Per-step wall time (ms) by pressure backend, full solver step at converged-benchmark sizes, one H200, fp64, same 10^{-6} pressure tolerance, Nu backend-identical. The execution mode of each backend is stated in the text.

problem	mesh	graph-MG	mixed	AmgX
Rayleigh–Bénard	h_{128}	88.0	90.4	85.3
	h_{256}	65.7	65.9	62.6
conjugate cavity	m2	43.1	44.3	41.6
	m3	44.1	44.6	43.9
cavity 2D	h_{128}	14.9	27.4	15.3
	h_{256}	18.6	30.0	22.0
cavity 3D	h_{64}	169.7	195.8	210.0
	h_{128}	1238.8	1234.8	1759.2

S3 Full distributed scaling study

This note holds the weak- and strong-scaling tables behind the main text for five problems: the three-dimensional differentially heated cavity, the unsteady Rayleigh–Bénard cell, the two-dimensional conjugate cavity, the three-dimensional conjugate heat sink and the single-cylinder engine. The measurement protocol they share is stated once in Supplementary Note S3.1. The per-problem subsections and their captions then give only what differs between cases.

S3.1 Shared protocol

Timing. Every table is a same-configuration measurement on H200 GPUs, median of three repetitions on exclusive nodes unless the caption says otherwise. “ms/step” is the convergence-free per-step throughput over a fixed step count: 200 steps for the two-dimensional transient problems, 3000 for the heat sink and 800 for the engine strong-scaling runs. “ms/PCG-it” is the per-Krylov machinery cost (matvec, halo exchange, V-cycle and all-reduce), timed with CUDA synchronization around the pressure solve alone. “it/outer” is the deterministic pressure (PCG) iteration count per SIMPLE outer step on the steady cavity. The pressure iterations of the transient problems are reported per step in some tables and per solve in others, and the two differ by the three RK substeps. “CG iters/step” and “pressure it/step” are per step; “CG iters/solve” and “pressure it/solve” are per solve. Each column heading, or the block label above the rows, states which it carries. The timing conventions of the main text (Section 4.2) apply throughout.

Degrees of freedom. The preconditioner A/B tables of Supplementary Notes S3.2 to S3.4 count *pressure* degrees of freedom. The three-dimensional conjugate tables of Supplementary Notes S3.5 and S3.6 and the size sweep of Supplementary Note S3.7 count *total* degrees of freedom over all fields, the coupled count $4N_{P_2} + N_{P_1}$ in three dimensions. In two dimensions the total is about 13 times the pressure count, since it also carries the two P_2 velocity components and the P_2 temperature.

Efficiency references. Strong-scaling speedup is $S = t_1/t_N$ on the whole-step ms/step, with parallel efficiency S/N and ideal $S = N$. Weak efficiency is t_{ref}/t_N at fixed load per GPU, with a flat ideal curve. The weak tables of the transient and conjugate problems (Supplementary Tables S9 and S13) reference it to two GPUs and also print the value against one GPU. The single-GPU run exchanges no halo and pays no cross-rank pressure penalty, so the one-to-two-GPU step is a communication onset rather than a scaling point, and the two-GPU column shows the multi-GPU trend. The cavity weak table (Supplementary Table S6) and the size sweep of Supplementary Note S3.7 reference weak efficiency to one GPU. Against one GPU the first doubling is the most expensive step

of every series, costing 0.61 to 0.82 across the Rayleigh–Bénard, conjugate-cavity, heat-sink and engine problems, while later doublings cost 0.71 to 0.92. Every weak efficiency quoted in this note names its reference.

Preconditioners. All comparisons are between our own preconditioners for the distributed pressure solve. One-level block-Jacobi applies the graph-multigrid V-cycle rank by rank with no coarse coupling between ranks. Nicolaidis deflation adds a one-degree-per-rank indicator coarse space; the steady-cavity tables compare block-Jacobi against it. The two-level preconditioner uses the per-rank V-cycle as smoother with a global aggregation coarse correction, solved redundantly on every rank, so its cost grows with the total problem; the Rayleigh–Bénard, conjugate-cavity, heat-sink and engine tables compare block-Jacobi against it. On the periodic- x , Neumann-pressure Rayleigh–Bénard operator the rank-wise deflation gives nothing, which is why only the global coarse correction appears there. At one GPU block-Jacobi and two-level are the same solve. We did not run a production distributed algebraic-multigrid library such as hypre BoomerAMG or AmgX under MPI on these operators, so no external distributed baseline is reported; the AmgX comparison in Supplementary Note S9 is single-GPU.

Pressure cap and capped work. The distributed transient runs cap the pressure solve at 300 iterations per solve, 900 per step over the three RK substeps. Where block-Jacobi reaches the cap its per-step work stops growing, its pressure is under-resolved, and its per-step time measures capped fixed work on a shrinking per-GPU load rather than convergence. Any efficiency or speedup it shows there is a capped-work artefact, not better scaling. The tables mark every capped entry with [†]. Block-Jacobi reaches the cap at four and eight GPUs in both weak series (Supplementary Table S9) and in the Rayleigh–Bénard block of the strong-scaling A/B (Supplementary Table S10), and at every multi-rank point of its conjugate-cavity block. Iteration counts are per step over the three RK substeps in the weak series and in the Rayleigh–Bénard block, where the cap shows as 900, and per solve in the conjugate-cavity block, where it shows as 300. The two-level preconditioner converges the pressure below the cap at every point.

Two sweeps. Two sweeps run here, and they answer different questions. The per-problem subsections (Supplementary Notes S3.2 to S3.6) hold each solver at its single-GPU memory ceiling and vary *only* the pressure preconditioner; they are the record behind Extended Data Fig. 1. Supplementary Note S3.7 fixes the preconditioner and varies the load, so it reports higher speedups. The three-dimensional cavity strong-scales $2.44\times$ on eight GPUs at 6.4×10^7 total DOF, against $1.72\times$ at the smaller size used for the A/B. The Rayleigh–Bénard and conjugate A/B tables quote single-GPU baselines re-measured under the frozen production configuration.

The two references cross-check each other. The eight-GPU Rayleigh–Bénard weak efficiency is 0.64 against two GPUs and 0.39 against one, and the size sweep reproduces that 0.39 independently as 0.38 (Supplementary Table S14). Absolute per-step times differ between the sweeps, which use different configurations and timing windows; normalized for the mesh the gap is 17% on the three-dimensional cavity. In two dimensions the meshes match exactly, since Supplementary Table S15 runs the same h_{1448} and h_{2040} problems as the A/B tables: the single-GPU step then differs by 3% on Rayleigh–Bénard (1150 against 1188 ms) and 14% on the conjugate cavity (901 against 1046 ms). The scaling trends agree.

S3.2 Cavity multi-GPU scaling: preconditioner A/B and coarse-space ablation

For the distributed cavity, weak scaling is in Supplementary Table S6 and strong scaling in Supplementary Table S7. Both compare block-Jacobi against Nicolaides deflation. Supplementary Table S8 repeats the strong-scaling A/B on a larger fixed problem, h_{161} , where the heavier per-GPU load holds the kernels compute-bound to higher rank counts and the deflation’s iteration saving turns into a larger wall-clock gain. A still larger h_{256} cannot be added to that comparison: it exceeds per-GPU memory below eight GPUs, so no single-node strong-scaling curve exists for it.

Coarse-space ablation. A richer affine coarse space $\{1, x, y, z\}$, four degrees per rank, is fragile. On the same fixed h_{128} cavity, deterministic and one repetition, it needs 283.5 pressure iterations per outer step at two GPUs against block-Jacobi’s 88.7, the large slabs giving an ill-conditioned coarse operator, and only 100.7 at eight against the indicator space’s 110.1. The one-degree-per-rank indicator space cuts iterations robustly at every rank count and at a fraction of the per-application cost.

Supplementary Table S6. Weak scaling of the distributed GNN pressure solve. 3D cavity, $Ra = 10^6$, $\approx 1.5 \times 10^6$ pressure DOFs per GPU (GPU-representative load; protocol of Supplementary Note S3.1). “bj eff” is block-Jacobi’s weak efficiency on ms/PCG-it against one GPU (np1 / npX); “it/o” is it/outer. The one-degree-per-rank Nicolaides deflation cuts pressure iterations 27–35% (growing with GPU count) at $\approx 10\%$ extra per-iteration cost, a net pressure-solve speedup of 22–29% (“Nic. net”). The per-GPU load follows the admissible cubic grids, so it is not constant (1.31–1.57M) and efficiency is reported both raw and per DOF (“bj eff/DOF”). The two differ most at four GPUs, which carries 17% less work per GPU than the baseline; per DOF the curve is monotone. A separate five-repetition sweep at the lighter 2×10^5 -DOF/GPU load gives 0.65 ($4.5 \rightarrow 6.9$ ms/PCG-it), the fixed per-iteration communication being less amortized there.

GPUs	pressure DOFs/GPU	bj it/o	Nic. it/o	bj ms/PCG-it	bj eff	bj eff/DOF	Nic. net
1	1.57M	29.0	29.0	6.30	1.00	1.00	–
2	1.51M	94.7	69.3	6.84	0.92	0.89	22%
4	1.31M	147.8	102.4	6.50	0.97	0.81	26%
8	1.50M	230.5	150.1	7.58	0.83	0.79	29%

Supplementary Table S7. Strong scaling of the distributed GNN solver. Fixed 3D cavity, $Ra = 10^6$, h_{128} ($\approx 1.57 \times 10^6$ pressure DOFs total), across one to eight GPUs; “wall-clock speedup” is the whole-SIMPLE-step speedup of Supplementary Note S3.1. The kernels strong-scale sub-linearly (block-Jacobi ms/PCG-it $6.29 \rightarrow 3.09$, $2.04\times$ on eight GPUs) while block-Jacobi’s pressure iterations grow steeply with rank ($29.0 \rightarrow 170.6$, a factor 5.9), so the whole-step speedup saturates, peaking near $1.6\times$ at four GPUs and regressing at eight. The Nicolaides deflation cuts iterations 29–35% at $\approx 10\%$ extra per-iteration cost and leads on iterations, absolute wall and speedup at every multi-GPU count (eight-GPU wall 750 against 829 ms). It turns over at eight as well ($1.92\times$ at four, $1.72\times$ at eight): the deflation raises the curve but does not remove the saturation, which is set by iteration growth this fixed 1.57×10^6 -DOF problem cannot amortize past four ranks.

GPUs	pressure DOFs/GPU	it/outer		ms/PCG-it		wall-clock speedup	
		bj	Nic.	bj	Nic.	bj	Nic.
1	1.57M	29.0	29.0	6.29	6.29	$1.00\times$	$1.00\times$
2	785k	88.7	63.1	4.28	4.71	$1.30\times$	$1.47\times$
4	393k	121.8	82.9	3.38	3.74	$1.64\times$	$1.92\times$
8	196k	170.6	110.1	3.09	3.41	$1.56\times$	$1.72\times$

Supplementary Table S8. Strong scaling on a larger fixed problem (h_{161} , 3.0×10^6 pressure DOFs). The sequence starts at two GPUs, so the speedup column is $S_{2 \rightarrow N} = t_2/t_N$ against the two-GPU run rather than the one-GPU reference used elsewhere (Supplementary Note S3.1). Median of five repetitions on exclusive H200 nodes, with the one degraded node excluded so the per-iteration timing is clean (within-cell spread $< 5\%$). h_{256} is omitted: it exceeds per-GPU memory below eight GPUs (see text), so a single-node strong-scaling curve cannot be formed. Relative to the main-text h_{128} problem, the larger problem keeps the kernels compute-bound longer (block-Jacobi machinery efficiency 47% vs 35% over two-to-eight GPUs), and the deflation’s iteration saving compounds into a larger wall-clock speedup than block-Jacobi.

GPUs	pressure DOFs/GPU	it/outer		ms/PCG-it		$S_{2 \rightarrow N} = t_2/t_N$	
		bj	Nic.	bj	Nic.	bj	Nic.
2	1.51M	94.7	69.3	6.83	7.27	1.00×	1.00×
4	755k	133.8	93.4	4.54	4.94	1.42×	1.42×
8	378k	188.8	126.9	3.63	4.07	1.47×	1.62×

S3.3 Unsteady Rayleigh–Bénard cell

The transient Rayleigh–Bénard cell has two tables: weak scaling at the single-GPU memory ceiling (Supplementary Table S9) and the fixed-problem A/B in which the two-level coarse correction recovers positive strong scaling (Supplementary Table S10, first block). Only the two-level preconditioner appears here, for the reason given in Supplementary Note S3.1. On a smaller fixed h_{724} problem (1.05×10^6 pressure DOF, 1.36×10^7 total, which one device runs whole) block-Jacobi behaves the same way. It runs at 349.1 ms/step on one GPU and 958.4, 862.9 and 915.1 ms on two, four and eight, a speedup of 0.36 to $0.40\times$ and a parallel efficiency falling from 0.18 to 0.05. The y -slab decomposition cuts the vertical pressure coupling, which aggravates the iteration growth.

Supplementary Table S9. Weak scaling of the two transient two-dimensional problems at the single-GPU memory ceiling. Both at a fixed $\approx 4.2 \times 10^6$ pressure DOF/GPU, the single-GPU ceiling, with $n \propto \sqrt{\text{NPROC}}$ and y -slab decomposition, over 200 steps. Weak efficiency is referenced to two GPUs, with the two-level figure against one GPU repeated in the last efficiency column (Supplementary Note S3.1). At the ceiling the global coarse correction is solved redundantly per rank, so its cost grows with the total problem, and that is the residual loss in both. *Rayleigh–Bénard*: $\Gamma = 2$ cell, $\text{Ra} = 10^7$, transient rotational-IPCS + RK3–CN, growing to 3.4×10^7 total pressure DOFs on eight GPUs; the $1.6\times$ one-to-two-GPU step is the onset, and the eight-GPU two-level efficiency is 0.64 against two GPUs (0.39 against one). *Conjugate cavity*: $\text{Ra} = 10^6$, $T_p = 0.1$, $k_s = k_f$, conjugate rotational-IPCS + RK3–CN, growing to 3.3×10^7 total on eight GPUs; the onset is $1.5\times$.

GPUs	DOF/GPU	ms/step		weak eff. (vs 2)		(vs 1)	CG iters/step	
		two-level	block-Jac.	two-level	block-Jac.	two-level	two-level	block-Jac.
<i>Unsteady Rayleigh–Bénard</i>								
1	4.20M	1150	1155	–	–	1.00	78	78
2	4.20M	1886	3794	1.00	1.00	0.61	127	699
4	4.19M	2156	4820	0.87	0.79	0.53	151	900 [†]
8	4.20M	2929	5259	0.64	0.72	0.39	178	900 [†]
<i>Two-dimensional conjugate cavity</i>								
1	4.17M	901	900	–	–	1.00	241	242
2	4.17M	1333	3330	1.00	1.00	0.68	104	655
4	4.16M	1884	4481	0.71	0.74	0.48	162	900 [†]
8	4.17M	2454	4539	0.54	0.73	0.37	179	900 [†]

[†] At the pressure cap (Supplementary Note S3.1), where block-Jacobi is $\approx 2\times$ slower on Rayleigh–Bénard and $1.9\text{--}2.5\times$ slower on the conjugate cavity with an under-resolved pressure.

Supplementary Table S10. A two-level coarse correction recovers strong scaling on four problems. Same-configuration A/B at fixed problem size over one to eight GPUs, protocol of Supplementary Note S3.1. Block-Jacobi’s pressure iterations grow steeply with rank. The global coarse correction holds that growth down, converges the pressure and recovers positive strong scaling on all four problems. Iteration counts are per step over the three RK substeps in the Rayleigh–Bénard and heat-sink blocks and per solve in the other two, as each block label states; [†] marks entries at the pressure cap, 300 per solve or 900 per step. *Rayleigh–Bénard*: $\Gamma = 2$, $Ra = 10^7$, fixed h_{1448} , $\approx 4.2 \times 10^6$ pressure DOFs, $\approx 5.5 \times 10^7$ total, $\approx 70\%$ of one H200, y -slab, 200 steps, all-eager; $\langle Nu \rangle$ identical across all eight runs. *Conjugate cavity*: $Ra = 10^6$, $T_p = 0.1$, fixed h_{2040} , $\approx 4.17 \times 10^6$ pressure DOFs, 5.4×10^7 total, y -slab, 200 steps, spreads $\leq 8\%$; two-level converges the pressure to 10^{-6} at every count, and the 3 ms gap at eight GPUs is within the run-to-run spread. *Heat sink*: fixed converged mesh, 1.52×10^7 total and 4.9×10^5 pressure DOFs, $Ra = 10^6$, x -slab, 3000 steps, all four rank counts in one exclusive-node job; θ_{src} identical across entries. *Engine*: fixed ceiling mesh, 6.2×10^7 total DOF, 104 GB = 73% of one H200, $Ra = 10^5$, $k_s/k_f = 10^3$, x -slab, 800 steps, one exclusive node; fire-deck and flux-weighted metal temperature identical across entries.

GPUs	ms/step		speedup t_1/t_N		pressure iterations	
	block-Jac.	two-level	block-Jac.	two-level	block-Jac.	two-level
<i>Unsteady Rayleigh–Bénard (per step)</i>						
1	1155	1150	1.00×	1.00×	78	78
2	2033	980	0.57×	1.17×	590	118
4	2179	868	0.53×	1.32×	898 [†]	148
8	1882	857	0.61×	1.34×	900 [†]	161
<i>Two-dimensional conjugate cavity (per solve)</i>						
1	900	901	1.00×	1.00×	81	80
2	1134	839	0.79×	1.07×	300 [†]	117
4	838	771	1.07×	1.17×	300 [†]	143
8	724	727	1.24×	1.24×	300 [†]	159
<i>Three-dimensional conjugate heat sink (per step)</i>						
1	819	819	1.00×	1.00×	84	84
2	931	742	0.88×	1.10×	251	104
4	1030	692	0.80×	1.18×	338	115
8	1314	695	0.62×	1.18×	469	126
<i>Single-cylinder engine (per solve)</i>						
1	4016	4016	1.00×	1.00×	27	27
2	2954	2573	1.36×	1.56×	116	40
4	2350	1797	1.71×	2.24×	159	47
8	2452	1494	1.64×	2.69×	223	51

S3.4 Two-dimensional conjugate cavity

Weak scaling at the memory-ceiling load is in Supplementary Table S9. The two-level versus block-Jacobi strong-scaling A/B for the conjugate cavity is the second block of Supplementary Table S10. Both hold the discretization and time step fixed and vary only the pressure preconditioner.

S3.5 Three-dimensional conjugate heat sink

The three-dimensional heat sink requires two-dimensional halo exchanges and a reduction at every Krylov iteration. A three-dimensional decomposition exchanges two-dimensional halo faces, with an all-reduce in every Krylov iteration. Grid convergence of the quantity of interest is in Supplementary Table S11, the same-discretization throughput in Supplementary Table S12, and weak and strong scaling in Supplementary Tables S10 and S13. The heat-sink block of the A/B table reports pressure iterations per step; block-Jacobi’s $84 \rightarrow 469$ per step is 28 to 156 per solve, so the 300-per-solve cap is never reached.

Supplementary Table S11. Grid convergence of the 3D conjugate heat-sink QoI. Source-averaged nondimensional temperature θ_{src} at thermal steady ($\text{Ra} = 10^6$, $k_s/k_f = 100$), without calibration to a target temperature. Coarse meshes staircase the thin conductor branches thicker and over-count the solid, biasing θ_{src} low; once h_{branch} resolves the branches the QoI converges to ≈ 2.36 (the two finest meshes agree to the four figures printed). The converged mesh is at the single-GPU memory ceiling, motivating the distributed solve.

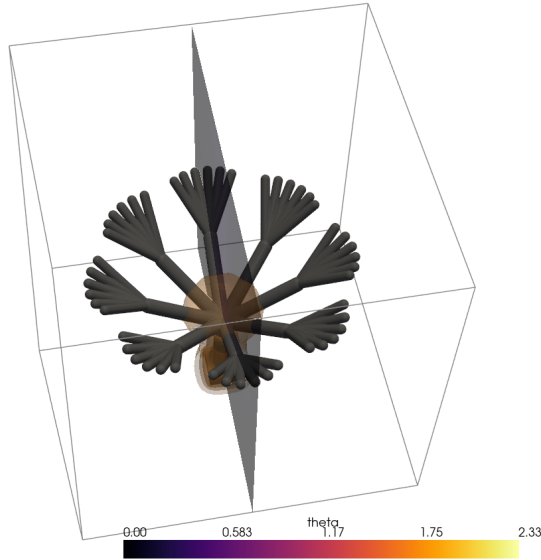
Mesh	total DOF	θ_{src}	branches resolved?
coarse	1.98×10^6	1.400	no (staircased)
fine	6.82×10^6	2.362	yes
finest	1.52×10^7	2.362	yes

Supplementary Table S12. Same-discretization throughput on the 3D conjugate heat sink (Tier B). Graph-multigrid on one GPU against PETSc on 32 CPU cores, same P₂–P₁ discretization, mesh, time step and 10^{-6} pressure tolerance, and the same 10^{-10} mass/diffusion tolerance in both implementations; median of three repetitions on an exclusive node. As on the engine, the mass tolerance has to be matched: leaving it mismatched at 10^{-7} on the GPU against 10^{-10} in PETSc inflates the ratio by 1.30–1.35 $\times$. The GPU driver runs eagerly; CUDA-graph capture was not evaluated for this case. Degrees of freedom are the coupled count $4N_{P_2} + N_{P_1}$. The ratio grows steeply with size because the GPU per-step cost rises by 7 $\times$ across a 31 $\times$ size range while the CPU cost rises by 88 $\times$.

Mesh	coupled DOF	GPU (ms)	CPU FEM (ms)	CPU/GPU
m1	3.92×10^5	228.9	3302.2	14.4
m2	1.98×10^6	316.8	22 609.4	71.4
m3	1.20×10^7	1598.5	289 614.5	181.2

Supplementary Table S13. Weak scaling of the two three-dimensional conjugate problems at the single-GPU memory ceiling. Two-level preconditioner, conjugate rotational-IPCS + RK3-CN, x -slab. Weak efficiency is referenced to two GPUs, with the value against one GPU in the last column (Supplementary Note S3.1). *Heat sink*: fixed $\approx 1.5 \times 10^7$ total DOF/GPU, the largest problem one H200 holds for this operator set, the mesh growing with NPROC to 1.13×10^8 DOF on eight GPUs; $Ra = 10^6$, 3000-step timing. The full-GPU load makes the one-to-two-GPU onset mild ($1.32\times$, against $3.45\times$ at a light 2×10^6 DOF/GPU load). Its eight-GPU weak efficiency at 1.1×10^8 DOF is 0.74 against two GPUs and 0.56 against one; the residual loss is the two-level global coarse correction growing with the total problem. *Engine*: fixed $\approx 6 \times 10^7$ DOF/GPU (104 GB, 73% of one H200), grown with NPROC; $Ra = 10^5$, median of three runs. It weak-scales comparably to the fluid-filled heat sink; a run at a lighter 1.6×10^7 DOF/GPU load weak-scales far worse (see text).

GPUs	total DOF/GPU	total DOF	ms/step	weak eff. (vs 2 GPU)	weak eff. (vs 1 GPU)
<i>3D conjugate heat sink</i>					
1	1.52e7	1.52e7	821.4	–	1.00
2	1.45e7	2.90e7	1081.1	1.00	0.76
4	1.45e7	5.81e7	1290.0	0.84	0.64
8	1.41e7	1.127e8	1462.6	0.74	0.56
<i>Single-cylinder engine</i>					
1	6.16e7	6.16e7	4016	–	1.00
2	6.26e7	1.25e8	4879	1.00	0.82
4	6.25e7	2.50e8	5548	0.88	0.72
8	6.17e7	4.94e8	6023	0.81	0.67



Supplementary Figure S2. Three-dimensional conjugate heat sink. Steady temperature field ($Ra = 10^6$, $k_s/k_f = 100$): a volumetric heat source at the base warms the branched conductor (isosurface), buoyancy lifts the hot plume through the branch canopy to the cooled lid, and the conductor spreads heat toward the cooled walls. Monolithic conjugate solve, graph-multigrid pressure, converged mesh (1.52×10^7 total DOFs).

S3.6 Distributed scaling of the engine solver

The engine problem is distributed with the same x -slab decomposition, graph-multigrid pressure solve and two-level coarse correction as the heat sink (Supplementary Note S3.5). A one-versus- N -GPU run reproduces the flux-weighted metal temperature to machine precision ($|\Delta\theta|/\theta < 10^{-11}$ across all rank counts and both preconditioners) and the fire-deck temperature (526.6 K) of Table S23, confirming the decomposition and the conjugate chamber-wall flux are exact at scale. On a fixed 6.2×10^7 -DOF engine mesh at the single-GPU memory ceiling (104 GB, 73% of one H200; Supplementary Table S10) the solver reproduces the heat-sink *strong*-scaling pattern: the two-level coarse correction reduces the pressure-iteration growth and recovers positive strong scaling, ahead of block-Jacobi at every count, the same coarse-correction recovery as the RBC, 2D-conjugate and heat-sink blocks of that table.

Weak scaling at the ceiling is in Supplementary Table S13. At a fixed $\approx 6 \times 10^7$ DOF/GPU (104 GB, 73% of one H200), the two-level solver weak-scales at 0.88 on four and 0.81 on eight GPUs against the two-GPU reference (0.72 and 0.67 against one GPU), matching or exceeding the fluid-filled heat sink at both counts (0.84 and 0.74 against two GPUs; 0.64 and 0.56 against one) despite the mostly-solid engine geometry ($\approx 81\%$ solid by volume). A run at a lighter 1.6×10^7 DOF/GPU (27% of the device) weak-scales far worse, 0.24 on eight GPUs against two: the efficiency here is governed by *filling the GPU* (amortizing the fixed per-step communication and global reductions against enough per-rank compute), not by the concentrated-flow geometry *per se*. The solver fits and runs the 2×10^8 -plus-DOF engine across four GPUs and beyond, reaching $\approx 4.9 \times 10^8$ DOF on eight. The strong-scaling recovery and the two-level-over-block-Jacobi ordering are robust across the three repetitions.

S3.7 Raw scaling versus problem size

Here the preconditioner is fixed and the load varies, across the three problems that share a common driver (the three-dimensional cavity, the unsteady Rayleigh–Bénard cell and the two-dimensional conjugate cavity). Degrees of freedom are counted in total here rather than in pressure unknowns, and weak efficiency is referenced to one GPU (Supplementary Note S3.1). Strong scaling improves monotonically with problem size, reaching $2.44\times$ on eight GPUs for the three-dimensional cavity at 6.4×10^7 DOF, because a larger per-GPU load keeps the kernels compute-bound out to higher rank counts. The end-to-end weak efficiencies here carry the pressure-iteration growth as well as the communication cost, so they sit below the machinery-only figure in Supplementary Table S6. Supplementary Figure S3 plots the three views of this sweep: weak efficiency at fixed DOF per GPU (Supplementary Table S14), strong speedup at matched total DOF (Supplementary Ta-

ble S15), and the two-to-eight-GPU speedup against problem size, which uses the ceiling meshes of Supplementary Table S16 alongside the matched ones.

Supplementary Table S14. Weak scaling at fixed total DOF per GPU ($\approx 37\text{M}$), all fields. Efficiency = t_1/t_N , single-GPU reference. Counts differ from the pressure-DOF sweeps of Supplementary Tables S6 and S7, which resolve the pressure solve alone on the same solver.

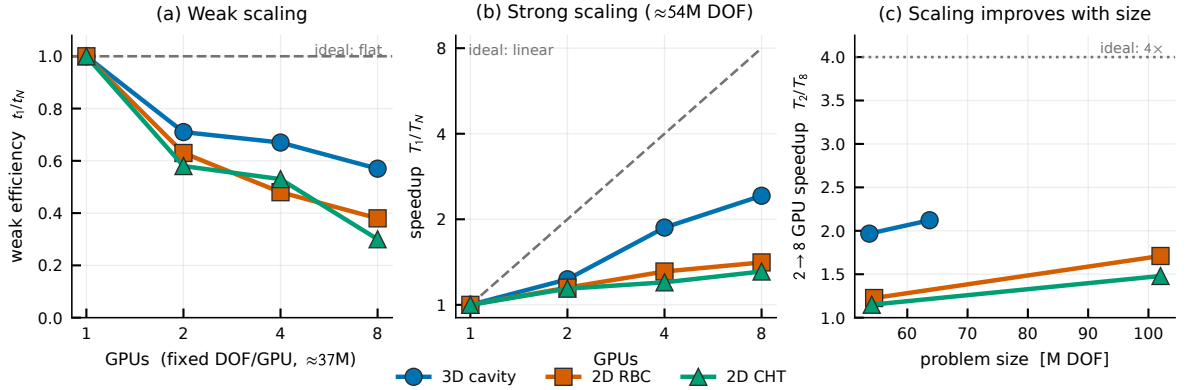
Problem	GPUs	Mesh	total DOF/GPU (M)	ms/step	Eff.
3D cavity	1	h_{120}	36.9	973 $\pm$ 5	1.00
	2	h_{151}	36.7	1371 $\pm$ 21	0.71
	4	h_{190}	36.6	1444 $\pm$ 46	0.67
	8	h_{240}	36.9	1708 $\pm$ 98	0.57
2D RBC	1	h_{1190}	36.8	807 $\pm$ 0	1.00
	2	h_{1683}	36.8	1281 $\pm$ 3	0.63
	4	h_{2380}	36.8	1672 $\pm$ 31	0.48
	8	h_{3366}	36.8	2116 $\pm$ 867	0.38
2D CHT	1	h_{1680}	36.7	707 $\pm$ 4	1.00
	2	h_{2380}	36.8	1212 $\pm$ 45	0.58
	4	h_{3360}	36.7	1335 $\pm$ 31	0.53
	8	h_{4760}	36.8	2389 $\pm$ 275	0.30

Supplementary Table S15. Strong scaling at matched problem size ($\approx 54\text{M}$ DOF), distributed GNN solver, H200 GPUs. Speedup = T_1/T_N , efficiency = speedup/ N . DOF are totals over all fields, not the pressure DOF of Supplementary Table S7.

Problem	Mesh	GPUs	total DOF (M)	total DOF/GPU (M)	ms/iter	Speedup	Eff.
3D cavity	h_{136}	1	53.7	53.7	1292 $\pm$ 3	1.00	1.00
		2		26.8	1052 $\pm$ 26	1.23	0.61
		4		13.4	692 $\pm$ 8	1.87	0.47
		8		6.7	534 $\pm$ 6	2.42	0.30
2D RBC	h_{1448}	1	54.5	54.5	1188 $\pm$ 2	1.00	1.00
		2		27.3	1033 $\pm$ 58	1.15	0.58
		4		13.6	907 $\pm$ 30	1.31	0.33
		8		6.8	840 $\pm$ 13	1.41	0.18
2D CHT	h_{2040}	1	54.1	54.1	1046 $\pm$ 3	1.00	1.00
		2		27.1	914 $\pm$ 37	1.14	0.57
		4		13.5	875 $\pm$ 45	1.20	0.30
		8		6.8	799 $\pm$ 26	1.31	0.16

Supplementary Table S16. Strong scaling at the largest meshes (near the single-GPU memory ceiling). RBC/CHT use a 2-GPU baseline (their meshes OOM on one GPU), so their efficiencies are not comparable to the single-GPU-referenced rows above. DOF are totals over all fields.

Problem	Mesh	GPUs	total DOF (M)	total DOF/GPU (M)	ms/iter	Speedup	Eff.
3D cavity	h_{144}	1	63.7	63.7	1404 $\pm$ 0	1.00	1.00
		2		31.8	1218 $\pm$ 5	1.15	0.58
		4		15.9	769 $\pm$ 10	1.83	0.46
		8		8.0	576 $\pm$ 7	2.44	0.30
2D RBC	h_{1980}	2	102.0	51.0	1887 $\pm$ 105	1.00	1.00
		4		25.5	1206 $\pm$ 10	1.56	0.78
		8		12.7	1106 $\pm$ 70	1.71	0.43
2D CHT	h_{2800}	2	102.0	51.0	1563 $\pm$ 66	1.00	1.00
		4		25.5	1177 $\pm$ 114	1.33	0.66
		8		12.7	1057 $\pm$ 284	1.48	0.37



Supplementary Figure S3. Scaling of the distributed solver against problem size. The three problems that share a common driver: the 3D cavity, the unsteady 2D Rayleigh–Bénard cell and the 2D conjugate cavity. Degrees of freedom are counted in *total* here and weak efficiency is referenced to one GPU, the conventions of this subsection rather than of the A/B tables above. **(a)** Weak efficiency t_1/t_N at a fixed ≈ 37 M DOF per GPU; the ideal curve is flat (Supplementary Table S14). **(b)** Strong-scaling speedup T_1/T_N at a matched ≈ 54 M total DOF; the ideal is linear (Supplementary Table S15). **(c)** The 2 $\rightarrow$ 8 GPU speedup against total problem size, from the matched-size and ceiling meshes (Supplementary Tables S15 and S16). For every problem the speedup rises toward the ideal 4 $\times$ as the mesh grows, because a larger per-GPU load keeps the kernels compute-bound out to higher rank counts. Okabe–Ito colours with a distinct marker per problem, so identity does not rest on colour alone.

S4 Multigrid as preconditioner versus solver

AI4PDEs [2] drives the pressure Poisson equation with a *standalone* multigrid iteration, a sawtooth cycle with Jacobi smoothing. We use the same graph multigrid as the *preconditioner* of an outer conjugate gradient. On the steady cavity the standalone multigrid

reaches the same Nusselt number, but its efficiency as a solver degrades with mesh refinement while the Krylov wrapper holds.

Notation. Write $S = G^\top \text{diag}(r_{AU}) G$ for the pressure Schur complement of the P_2 - P_1 saddle point (the SIMPLE pressure-correction operator, with $r_{AU} = 1/a_P$ the reciprocal *unrelaxed* momentum diagonal), K_p for the P_1 stiffness Laplacian on which the algebraic-multigrid hierarchy is built, and B for one multigrid V-cycle (the preconditioner, $B \approx S^{-1}$). The stationary *solver* iteration is $p \leftarrow p + \omega B(f - Sp)$ with under-relaxation ω and iteration matrix $E = I - \omega BS$; it converges iff the spectral radius $\rho(E) < 1$, i.e. $\omega < 2/\lambda_{\max}(BS)$. We write the spectral condition number $\text{cond}(A) = \lambda_{\max}(A)/\lambda_{\min}(A)$.

The V-cycle is symmetric positive definite. Before comparing, we verify the preconditioner is symmetric positive definite (SPD), so that any failure of the stationary iteration is a spectral-conditioning effect and not an asymmetry artefact. The smoothed-aggregation hierarchy sets restriction equal to the transpose of prolongation to machine precision ($\max |R - P^\top| / \max |P| = 0$ at every level), the pre- and post-smoothing counts are equal, and the smoother (weighted Jacobi or Chebyshev) is symmetric; the numerical test $\langle Bx, y \rangle = \langle x, By \rangle$ holds to 3×10^{-16} (Jacobi) and 1×10^{-16} (Chebyshev) for random x, y . Consequently BS has real positive eigenvalues and $\rho(E) < 1$ is achievable for small enough ω .

The graph execution reproduces the cycle it represents. Correct operators and an SPD preconditioner do not yet establish that the graph *execution* performs the intended multigrid action. A reference V-cycle written in `scipy` is run on the hierarchy the graph object itself holds, extracted from its own layers, with the same weighted-Jacobi smoother, the same $n_{\text{pre}}/n_{\text{post}}$, and the same coarsest-level treatment. The two agree to a relative L^2 difference of 1.5×10^{-16} to 3.8×10^{-16} , which is fp64 round-off over the several dozen sparse applications a cycle performs. This holds across $\omega \in \{0.5, 0.7, 0.9\}$ and $(n_{\text{pre}}, n_{\text{post}}) \in \{(2, 2), (1, 1), (3, 1)\}$ on 32^2 , 64^2 and 128^2 Poisson operators of four and five levels, so it is a property of the execution and not of one tuned configuration. Note that PyAMG’s own preconditioner is not the right reference here: it defaults to Gauss–Seidel and solves the coarsest level directly, whereas our cycle relaxes it, so a disagreement there would say nothing about the graph path.

One practical consequence surfaced in the same test. Rebuilding the smoothed-aggregation hierarchy from the identical operator does not return the identical hierarchy: a fresh build differs from the one in use by up to 6.5×10^{-3} relative in the coarse operators. The outer Krylov solve converges to the same answer regardless, since any of these is a valid preconditioner, but reproducing a run to the last bit requires caching the

hierarchy rather than rebuilding it. The identity above is stated against the hierarchy actually held, which is why it reaches round-off.

Stationary multigrid on the Schur complement diverges. Replacing the conjugate gradient by the stationary iteration E on the *same* operator S and the *same* V-cycle B diverges at every under-relaxation tried ($\omega \in \{1, 0.5, 0.3, 0.1, 0.05\}$): the Nusselt number overshoots and the pressure residual blows up (to $\sim 10^{62}$), smaller ω merely delaying the onset. The cause is $\lambda_{\max}(BS)$: the multigrid is built on the constant-coefficient K_p but applied to the wall-zeroed, spatially varying S , so BS carries large outlying eigenvalues that violate $\omega \lambda_{\max} < 2$. The conjugate gradient remains convergent on this SPD problem and needs no such estimate, so it avoids selecting a stationary relaxation from this spread: it is optimal over the Krylov subspace for any SPD operator. A fixed-step iteration is not.

Standalone multigrid on the Laplacian projection converges, but weakly. The faithful Neural Physics analogue instead solves a constant-coefficient projection $A_p \delta p = f$ with $A_p = c K_p$, so that the multigrid solves its *own* operator and is robust. The scalar c is fixed by the Rayleigh quotient $c = \langle v, Sv \rangle / \langle v, K_p v \rangle$ (a single mean r_{AU} misses the $G^T G$ -versus- K_p operator scale $\sim h^2$ and under-scales the correction by $\sim 10^4$); this gives a relative operator match $\|Sv - c K_p v\| / \|Sv\| = 0.15$. Solved this way, the standalone multigrid converges to the *same* Nusselt number as the preconditioned conjugate gradient at every Rayleigh number and mesh, in essentially the same number of outer SIMPLE iterations (within $\sim 10\%$; Supplementary Table S17). The cost is the inner solve: the V-cycle is a weak stationary iteration whose convergence factor degrades toward ≈ 0.94 at h_{128} , so its cycle count per step *grows with refinement* (23.8–48.6 at h_{64} but ≈ 228 at h_{128}). The conjugate gradient collapses the identical V-cycle to 10.5–46.3 iterations and stays mesh-robust. The end-to-end wall penalty of the standalone solver therefore grows from $\approx 2.5\times$ at h_{64} to $\approx 17\times$ at h_{128} .

Supplementary Table S17. Standalone multigrid (Neural-Physics-style) versus multigrid-preconditioned conjugate gradient on the 2D cavity pressure solve, settings identical except the pressure mode and, for the standalone solver, the substituted constant-coefficient operator $A_p = c K_p$ (relative operator match 0.15). The two solvers agree on Nu to the four figures printed, so one column serves both. Wall ratio is standalone-solver over preconditioned-CG end-to-end.

Ra	mesh	converged		outer SIMPLE iters	inner iterations/step		wall-clock ratio
		Nu			solver	CG	
10^4	h_{64}	2.2450		5200	48.6	46.3	2.3
10^5	h_{64}	4.5236		3000	28.2	11.7	3.0
10^6	h_{64}	8.8466		5200	23.8	10.5	2.5
10^4	h_{128}	2.2451		13100	227.6	21.3	17.4

Conclusion. The graph multigrid is an effective *preconditioner* but a weak standalone *solver*: as a stationary solver it either diverges (on the exact variable-coefficient Schur complement) or converges only at a mesh-growing iteration count (on the constant-coefficient Laplacian projection), a penalty that grows with resolution. We therefore use the multigrid as a Krylov preconditioner throughout the steady-cavity benchmark, in both the CNN and the GNN, rather than as the standalone solver of Chen et al. [2]. (The CNN’s explicit transient Rayleigh–Bénard march is the one path that uses a standalone multigrid projection, Section 4.3.)

S4.1 Preconditioner strength: graph multigrid versus PETSc GAMG

The per-step throughput of the conjugate solver, graph multigrid on one GPU against PETSc CG + GAMG on 32 CPU cores, is tabulated in Supplementary Table S22. That ratio reflects the linear-solver backend and hardware (Tier B). We compare pressure-iteration counts at the matched residual tolerance to assess differences in preconditioner effectiveness. Matching the tolerance alone does not establish this: two solvers can both reach 10^{-6} with one needing an order of magnitude more cycles than the other, as Supplementary Table S17 shows for the standalone multigrid. Supplementary Table S18 lists preconditioned-CG iterations per pressure solve for the two implementations on the identical anchored Laplacian K_{anc} , at the identical true-residual tolerance 10^{-6} , averaged over the transient ($\text{Ra} = 10^6$, $T_p = 0.1$, $k_r = 1$, RK3–CN, fp64). A separate campaign reproduces the m_1 – m_5 counts to within 0.2 iterations. Both implementations cold-start each solve from zero (the projection solves for the pressure *increment*, so there is no warm-start asymmetry), and both stop on the unpreconditioned residual $\|b - Ax\|/\|b\|$. PETSc’s CG + GAMG needs fewer iterations at every mesh; the graph multigrid needs 17–42% more, the gap widest (42%) at m_6 , the resolution at which the throughput ratio of Supplementary Table S22 is quoted. The imbalance does not favour the GPU: the GPU run reaches the same tolerance while doing more Krylov work than the CPU run, and normalizing to a common iteration count would widen the per-step gap rather than narrow it. The counts compare conditioning, not time; the two hierarchies are different algebraic-multigrid implementations at their library defaults. What remains in the ratio is the combined effect of the two libraries’ per-cycle cost and the execution device (Tier B). The hierarchy is a standard smoothed-aggregation construction, and the contribution is its execution as fixed-weight message-passing layers on the GPU, not the hierarchy itself (Section 4.3).

Supplementary Table S18. Preconditioned-CG iterations per pressure solve at matched tolerance. Same operator, same 10^{-6} true-residual stopping criterion, in-solver average over the transient. Both grow only mildly over a $31\times$ range of problem size ($8.8 \rightarrow 11.5$ and $10.3 \rightarrow 16.3$), far from the growth of an inadequate preconditioner, for which Supplementary Table S17 is the contrast. We make no claim as to which library’s multigrid is superior.

Mesh	P ₂ DOF	Graph MG (GPU)	CG + GAMG (CPU)
m1	11 963	10.3	8.8
m2	23 899	10.9	8.9
m3	46 929	12.5	9.0
m4	94 291	11.2	9.3
m5	185 949	12.9	9.9
m6	373 073	16.3	11.5

S4.2 Toward isolating the hardware: the identical graph multigrid on CPU and GPU

The comparison above exchanges two multigrid libraries along with the hardware. To reduce that confound we ran the identical solver on the CPU backend and on the GPU, changing only the execution device: the same smoothed-aggregation hierarchy, Chebyshev smoother, fixed-weight message-passing code, conjugate configuration and step count. The CPU run uses 32 threads, matching the allocation of Supplementary Table S22. CUDA-graph capture is part of the GPU path. It is unavailable on the CPU backend and is disabled there. Extended Data Table 3 gives the per-step ratio at fixed algorithm: $4.6\times$ at the coarsest mesh and $21.4\times$ at the production mesh. Those GPU runs reproduce the per-step column of Supplementary Table S22 ($40.5 \rightarrow 60.75$ ms) to within a few per cent. They are separate GPU runs, so that agreement is a reproducibility check and not an identity. At the production mesh the same-code ratio $21.4\times$ is close to the $23.9\times$ same-method ratio against PETSc CG + GAMG in Supplementary Table S22, but the two come from separate campaigns and use different hierarchies and iteration counts (Supplementary Table S18), so they do not separate device from backend numerically. The CPU execution of this solver is also not an optimized CPU implementation. The tuned CPU reference remains the PETSc CG + GAMG run of Supplementary Table S22, and this measurement isolates the device at fixed code as a complement to that baseline.

S5 Differentiating the solver

Expressing the entire solver as message passing in one automatic-differentiation framework makes the coupled step differentiable, where a call into a conventional sparse-linear-algebra library would require a custom adjoint or an implicit-differentiation interface. This note verifies the gradient on the conjugate cavity, asks whether it is the sensitivity

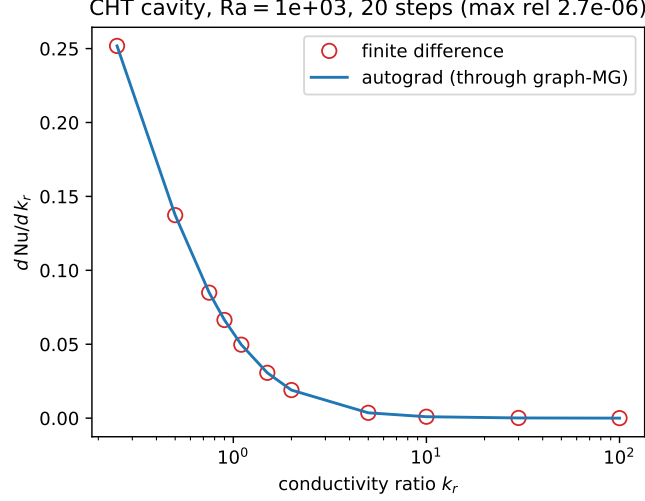
of the discrete solution, carries the implicit rule through the coupled step, and records cost and memory against central finite differences and against pyadjoint. Two routes to the gradient appear throughout and are reported separately. *Unrolled* reverse mode differentiates the executed program and stores every linear-solver iterate on the tape. *Implicit* differentiation differentiates the residual of each linear solve at its solution, stores one residual, and takes one transposed solve. Measurements on the single thermal step and on the coupled solver are also kept apart, because their costs and memory are not comparable.

Four limits apply to everything in this note. The multigrid hierarchy is constructed once, outside the differentiated graph. Unrolled reverse mode differentiates its application, whereas implicit differentiation uses a residual-based backward rule and a transposed solve without differentiating the Krylov iteration history. The demonstration is single-GPU, because the distributed halo exchange is in place and breaks the tape. The runs use fixed linear-solve iteration counts rather than the production tolerance-based stopping criterion, which makes the forward map a smooth function of the parameter; transferability to the tolerance-controlled production configuration is not established here. And the production driver’s optional Picard momentum path is a nonlinear fixed point rebuilt each iteration, whose adjoint is that of the nonlinear system rather than of each inner solve; it is not addressed here.

S5.1 Verification of the differentiated program

The discretization, scheme and mesh are held fixed. We take the solid–fluid conductivity ratio k_r as a differentiable scalar, using that the thermal stiffness is exactly affine, $K_T(k_r) = K_{\text{fluid}} + k_r K_{\text{band}}$. We run the full coupled rotational-IPCS + RK3–CN solve on a short window, and read the hot-wall Nusselt number as a linear functional $\text{Nu} = g^\top T$ of the temperature vector. Unrolled reverse-mode differentiation runs through the *entire* coupled step: the advection assembly, the energy solve, and the graph-multigrid pressure V-cycle, which the gradient reaches through its buoyancy-driven right-hand side. The sensitivity $d\text{Nu}/dk_r$ agrees with a central finite difference to a relative 10^{-9} – 2.7×10^{-6} across $k_r \in [0.25, 100]$ (Supplementary Figure S4); the largest relative errors are the finite-difference floor at the strongly conducting end, where the sensitivity itself is $\mathcal{O}(10^{-5})$. Every linear solve, pressure, momentum and energy, uses a fixed iteration count. The forward map is therefore a smooth deterministic function of k_r , and automatic differentiation and finite differencing see the identical discrete map. The momentum, pressure and correction operators are k_r -independent and reuse the production solvers unchanged, so the gradient path they carry is differentiability with respect to their right-hand side, which is the property the graph-multigrid V-cycle is built to have. This is a verification of differentiability, not an inverse-design or optimization result: the same gradient would

drive PDE-constrained optimization or parameter estimation with no hand-derived adjoint. It is also a check on the differentiation and not on the sensitivity. Both routes evaluate the same fixed-iteration program, and the two can differ from the sensitivity of the converged discrete solution; the next subsection measures by how much.



Supplementary Figure S4. The graph-multigrid solver is differentiable end-to-end. Sensitivity of the conjugate hot-wall Nusselt number to the conductivity ratio, $d\text{Nu}/dk_r$, by unrolled reverse-mode automatic differentiation through the full coupled solve (line) versus a central finite difference (markers), conjugate cavity at $\text{Ra} = 10^3$. The gradient backpropagates through the graph-multigrid pressure V-cycle via its buoyancy-driven right-hand side; a conventional linear-algebra library call is opaque to that tape.

S5.2 Convergence of the unrolled derivative to the discrete sensitivity

On one implicit Crank–Nicolson conjugate step assembled from the production operators the thermal operator is affine in k_r , so the converged-solution derivative follows in closed form from the adjoint: with $\mathbf{A}(k_r)x = b(k_r)$ and $\text{Nu} = g^\top x$, solving $\mathbf{A}^\top \lambda = g$ gives $d\text{Nu}/dk_r = \lambda^\top (\partial_{k_r} b - \partial_{k_r} \mathbf{A} x^*)$. Both solves are direct, so the reference carries no iteration error; it agrees with a central difference of the converged solve to 2.3×10^{-8} .

Against that reference, differentiating through n fixed Krylov sweeps converges slowly and not monotonically. At $k_r = 2$ the error is 100% at $n = 8$, 114% at $n = 20$, and first reaches 10^{-6} at $n = 80$; at $k_r = 100$ it passes through a factor of 297 at $n = 50$ and 13.5 at $n = 80$ before converging by $n = 200$. On the next mesh up it is worse, peaking at 2.9×10^4 at $n = 80$ and not converging until $n = 400$. The forward residual is not a usable guide: on that finer mesh at $n = 80$, $k_r = 100$, it reads 2.6×10^{-6} while the derivative is wrong by four orders of magnitude. Throughout, a central difference through the same

program agrees with the unrolled derivative to between 10^{-5} and 10^{-8} , so the two never disagree in a way that would reveal the error.

S5.3 Implicit differentiation

Single thermal step. Taping the solver is not necessary. Writing the solve as $r(x, \theta) = A(\theta)x - b(\theta) = 0$ and differentiating the residual at fixed x gives $dL/d\theta = -\lambda^\top \partial_\theta r$ with $(\partial_x r)^\top \lambda = \partial_x L$, so the backward pass needs one transposed solve and no iterate history. On the same step this changes the picture: at $n = 20$ the error falls from 114% to 9.1×10^{-7} , 10^{-6} is reached at $n = 20$ rather than $n = 80$, the error decreases monotonically with the forward residual, and the floor is 6.5×10^{-15} against the unrolled 5.3×10^{-11} . The transposed solve must be converged in its own right: tying its iteration count to the forward's leaves the error at 10^{-4} or worse, and at 1.9×10^4 nodes it needs about 60 iterations to reach 10^{-8} .

Coupled step. The rule is carried through all six linear solves of each Runge–Kutta substage. Only the thermal operator carries k_r , so only it needs the general rule; the two momentum, the pressure and the two corrector solves have operators fixed at setup, and for those the sensitivity to the right-hand side is one transposed solve with no residual to tape. We compare the forward outputs, converged gradients, and gradients at the prescribed forward-iteration count. By the *converged sensitivity* we mean the sensitivity obtained once the linear subsolves are converged over the specified time window. The windows here are twenty and forty steps, so it is a converged-subsolve reference over that window and not a fully equilibrated steady-state sensitivity. The forward is untouched, so Nu agrees between the two routes to the solver's own reproducibility floor, about 10^{-14} , which is set by the advection assembler's atomics and not by the rule. At converged counts and $k_r = 2$ the two gradients agree with each other to 2×10^{-12} over a twenty-step window and 1×10^{-12} over forty, and each sits within 1.2×10^{-11} of a central finite difference at twenty steps and 2.9×10^{-9} at forty, the finite difference being the looser of the two comparisons throughout. At the prescribed forward count $n = 40$ of this experiment, where the thermal residual is 3.6×10^{-6} and the solve is by construction not converged, the implicit gradient is 5.1×10^{-4} from the converged sensitivity while the unrolled one is 25% from it at $k_r = 100$, and 4.5×10^{-8} against 3.9×10^{-6} at $k_r = 2$. It does not make the gradient exact: the implicit backward is evaluated at the inexact forward state x_n , which still enters the term containing $\partial_{k_r} A$, so a residual-dependent error remains and is what the 5.1×10^{-4} measures. What the rule removes is differentiation through the truncated iteration, and that is the larger of the two errors by three orders of magnitude here. The two routes are not run at equal cost per solve: the transposed solves use four

times the forward’s iteration count, for the reason given above, and that is included in the cost figures of Supplementary Note S5.4.

S5.4 Cost and memory

Single thermal step, unrolled against implicit. Across a mesh sequence from 8.4×10^3 to 1.2×10^6 nodes the implicit gradient costs 2.0 to 2.9 forward solves and is flat in problem size, whereas the unrolled gradient rises from 7.4 to 29.3 because the iteration count grows with the mesh. Peak memory at the largest size is 10.7 GiB for the implicit route against 35.5 GiB for the unrolled one. Neither route reached the 141 GB limit on this mesh sequence, so where the implicit route stops is not established here; the unrolled one, growing by about $3.9\times$ per refinement, would stop at the next size. These figures are for the single thermal step and are not comparable with the coupled multi-step costs and tape sizes below, or with Supplementary Table S19.

Coupled step, unrolled against implicit. Over the coupled window of Supplementary Note S5.3, both routes measured in the same runs, the implicit sweep takes 4.8 to 4.9 forward solves against the unrolled 5.4 to 5.7. Main-text Figure 2c reports 4.9 and 5.6 for the parameter sweep, timing all three routes in one run at the same step and iteration counts, so the two studies agree. The implicit peak memory is flat at 2.720 GiB from 40 to 320 iterations while the unrolled tape grows $7.56 \rightarrow 22.14 \rightarrow 41.57$ GiB. At 2000 iterations the unrolled route exhausts the same 141 GB H200 and the implicit one is unchanged.

Coupled step, against central finite differences. We next compare the cost of reverse-mode differentiation with central finite differences. All routes are the same solver on the same GPU, differing only in how the derivative is obtained. To vary the parameter count we cut the solid band into M layers, each carrying its own conductivity, so that $K_T(\mathbf{k}) = K_{\text{fluid}} + \sum_{j=1}^M k_j K_{\text{seg},j}$ keeps the same affine structure and $M = 1$ recovers the scalar case above. A layered conductor is the natural several-parameter design problem for this geometry. At every M the gradient is verified componentwise against central differences before its cost is recorded, agreeing to 7.8×10^{-9} – 5.3×10^{-8} , and at $M = 1$ the layered operator reproduces the scalar operator to 0 in Nu and 1.4×10^{-14} in the gradient.

Figure 2c in the main text gives the result, a crossover. One unrolled reverse sweep costs 5.6 forward solves and one implicit sweep 4.9, and both costs are independent of how many parameters the gradient carries: they move by 2.4% and 1.6% while the parameter count rises $32\times$. Central differences instead cost $2M$ forward solves. At a single parameter finite differencing is $2.6\times$ cheaper, so there is no reason to prefer automatic

differentiation on cost grounds; the two break even near $M = 3$, and by $M = 32$ the unrolled sweep is $11.3\times$ cheaper and the implicit one $13.1\times$. Differentiating the solver pays on design problems with many parameters, not on single sensitivities. The flat scaling rests on one implementation detail: the operator entries depend on $\mathbf{k}$ alone and not on the vector being multiplied, so they are formed once per solve and the matrix-vector product stays independent of M . Assembling per layer inside the product would have made the measured reverse-sweep cost grow with M as an artefact of the assembly.

S5.5 Comparison with pyadjoint on DOLFINx

Finite differences are the alternative that needs no adjoint at all. We also compare with pyadjoint on DOLFINx using the same conjugate thermal problem [3, 4] (pyadjoint 2026.4.1 with dolfinx-adjoint 0.2.1; exact versions and the two local compatibility patches are in Supplementary Table S32). For this comparison we restrict to the conjugate *thermal* step, the part the conductivity enters, and give both implementations the same discrete problem rather than two similar ones: the same mesh, operators assembled from the same forms, Crank–Nicolson over the same window, the same hot-wall functional and the same linear-solve tolerance. The timed comparison in Supplementary Table S19 is between our *unrolled* reverse mode and pyadjoint; the implicit route enters only as an accuracy reference. The two Nusselt numbers agree to 8×10^{-6} – 1×10^{-4} , so the times are comparable. The implicit gradient and pyadjoint’s agree to 4×10^{-7} – 2×10^{-5} across the table, two independent rules for the same converged sensitivity. The unrolled gradient sits about 10^{-3} from both, which is the truncation it differentiates rather than an error.

Supplementary Table S19 separates into two effects that point in opposite directions. Our unrolled differentiation is the more expensive of the two: taping the solver costs 5.2–6.9 forward solves, where pyadjoint costs a near-constant 1.7. The difference follows from what each does. Unrolling differentiates through the linear solver, iteration by iteration; pyadjoint differentiates the linear solve analytically and takes one extra adjoint solve, which is what $1.7\times$ buys. Our *forward* is the faster of the two by 7 to $23\times$, and that advantage more than covers the overhead, so the delivered gradient is $2.2\times$ faster at the coarsest mesh and $5.9\times$ at the finest, widening with problem size exactly as the throughput comparisons do. Both frameworks are nearly flat in the parameter count, as reverse mode should be.

In the unrolled comparison, storing the linear-solver iteration history limits the largest tractable problem. The tape holds every solver iterate, so the requirement grows with steps $\times$ iterations $\times$ nonzeros: 3.7 GiB at 1.2×10^4 degrees of freedom and 65.3 GiB at 9.4×10^4 . The tape grows by about $2.6\times$ for each doubling of the mesh, faster than the problem itself, so the next mesh in the sequence needs roughly 170 GiB and does not fit on the 141 GB H200. That is the largest unrolled run reported here. pyadjoint stores

one state per step and has no such ceiling here. The limit belongs to unrolling and not to differentiating the solver as such: the implicit route of Supplementary Note S5.3 stores one residual per solve, and its memory is independent of the iteration count (2.720 GiB on the coupled step, and 10.7 GiB against the unrolled 35.5 GiB at the largest single-step size, Supplementary Note S5.4). In this comparison pyadjoint avoids the iteration-history storage that unrolling requires. The implicit implementation also avoids that storage, but its runtime is not compared with pyadjoint here. The case for differentiating the solver is that the gradient exists at all, on any operator the solver can apply, with no adjoint to derive or maintain.

Supplementary Table S19. Unrolled reverse-mode AD against pyadjoint on DOLFINx, same discrete problem. Conjugate thermal step, Crank–Nicolson, 20 steps, both to a 10^{-8} linear-solve tolerance in the forward and adjoint solves alike; median of three timed runs. “Ours” is the unrolled route, with every linear-solver iterate stored on the tape; the implicit route is not timed in this table. Ours on one H200, pyadjoint on 32 CPU cores; doubling that allocation from 16 cores changed the pyadjoint time by -5 to $+3\%$, so it does not thread-scale here either. M is the number of independently varying layer conductivities, and every row here is $M = 1$. “Overhead” is the gradient cost divided by that implementation’s own forward cost and is the price of the differentiation machinery; the ratio column is the delivered gradient wall clock. The meshes at 1.9×10^5 and 3.7×10^5 DOF have no entry here. Repeating every row at $M = 8$ moves no column by more than 4.9%, so both implementations are flat in the parameter count; those rows are not shown.

DOF	ours (GPU, unrolled)				pyadjoint (CPU)		
	fwd (s)	grad (s)	ovh	tape (GiB)	grad (s)	ovh	ratio
11963	0.24	1.23	5.23	3.7	2.75	1.64	2.24
23899	0.33	1.72	5.26	9.7	5.13	1.62	2.98
46929	0.45	2.54	5.60	26.0	10.09	1.73	3.97
94291	0.57	3.99	6.93	65.2	23.50	1.79	5.90

S6 Conjugate heat transfer: additional benchmarks and transient development

The primary conjugate benchmark (the transient conjugate cavity of Khatamifar et al. [5]) and the same-method throughput result are reported in the main text (Section 2.5). The supporting validation is here: a second published benchmark with a two-dimensional fluid–solid interface, the effect of partition thickness, and the grid-converged transient development.

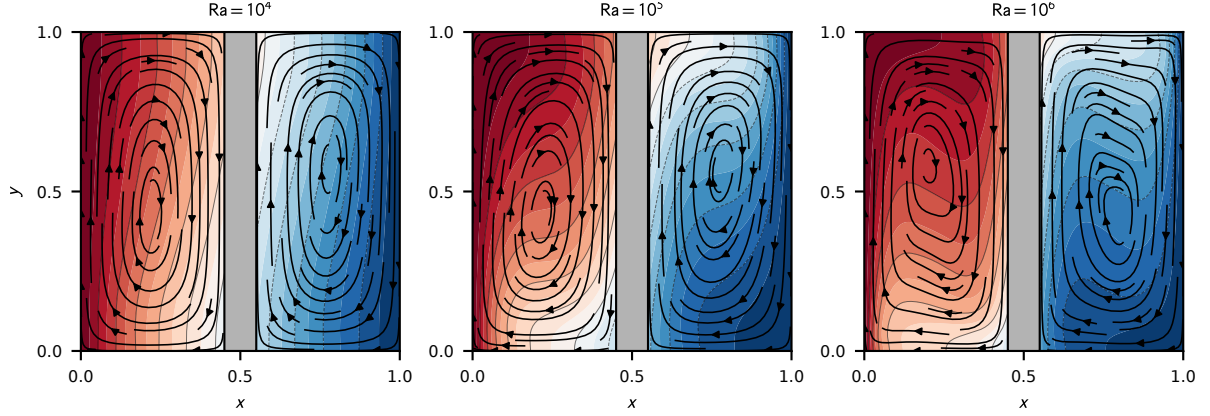
Second benchmark: a centred conducting body. The partition spans the cavity, so heat crosses it in essentially one dimension. A more stringent conjugate test is a solid that the flow must route *around*, with a genuinely two-dimensional interface. We

therefore also validate against the classical centred-conducting-body benchmark of House et al. [6]: a differentially heated square cavity ($\theta = +\frac{1}{2}$ hot, $-\frac{1}{2}$ cold, adiabatic top and bottom) containing a centred square solid of side ζ and conductivity ratio $k_r = k_s/k_f$, solved with the identical monolithic conjugate formulation. Supplementary Table S20 reports the steady hot-wall Nusselt number for the five cases that span the benchmark: a moderate body ($\zeta = 0.5$) at $Ra = 10^5$ across the full conductivity range $k_r \in \{0.2, 1, 5\}$, and a large body ($\zeta = 0.9$, leaving only a 0.05-wide fluid gap on each side) at $Ra = 10^6$ for the insulating and conducting extremes. The agreement is close in four of the five cases, which match the reference to within 0.23%, and the strongly conducting large-body case ($\zeta = 0.9$, $k_r = 5$) to 1.6%. The streamfunction extremum agrees in parallel (within $\sim 1\%$ of the reference where tabulated), confirming the flow and not only the wall flux. Because the benchmark is steady with no volumetric source, the hot- and cold-wall Nusselt numbers must be equal; the solver returns them equal to four significant figures in every case ($|\text{Nu}_h - \text{Nu}_c|/\text{Nu} < 10^{-4}$), a near-closed global energy balance across the conjugate interface. The two $\zeta = 0.9$ cases are grid-converged across h_{160} and h_{240} (the insulating case changes by 0.12% between the two meshes). For the strongly insulating large body the solid conduction time scale is long. The body interior equilibrates far more slowly than the surrounding flow, so we advance the buoyant flow to steady with the graph-multigrid solver and obtain the equilibrated temperature by a direct steady solve of the conjugate energy equation on that flow; this is the steady end-state the time-marched field approaches, reached without resolving the slow solid transient.

Supplementary Table S20. Validation against the centred-conducting-body benchmark of House et al. [6]. Differentially-heated square cavity with a centred square solid of side ζ and conductivity ratio $k_r = k_s/k_f$, $Pr = 0.71$. “This work” is the steady hot-wall Nusselt number from the monolithic GNN conjugate solver (for $\zeta = 0.9$, the h_{240} mesh); the hot- and cold-wall values are equal to four significant figures ($|\text{Nu}_h - \text{Nu}_c|/\text{Nu} < 10^{-4}$), so a single Nu is reported.

ζ	Ra	k_r	This work (GNN)	House (1990)	diff.
0.5	10^5	0.2	4.626	4.626	+0.02%
0.5	10^5	1.0	4.506	4.506	+0.00%
0.5	10^5	5.0	4.319	4.322	−0.08%
0.9	10^6	0.2	2.408	2.402	+0.23%
0.9	10^6	5.0	3.806	3.868	−1.61%

Effect of partition thickness. A distinctive feature of the conjugate problem is that the partition reduces heat transport even when it is of equal conductivity ($k_r = 1$): the solid band carries no convection, so thickening it replaces convective transport with pure conduction over a wider strip. Doubling the partition to $T_p = 0.2$ lowers the grid-converged Nu from 1.059, 1.751 and 2.794 to 1.020, 1.432 and 2.123 at $Ra = 10^4$, 10^5 and



Supplementary Figure S5. Flow structure in the conjugate cavity (GNN, $k_r = 1$, $T_p = 0.1$). Isotherms (filled, hot left / cold right) with streamlines overlaid, at $Ra \in \{10^4, 10^5, 10^6\}$; the grey band is the solid partition, in which $\mathbf{u} \equiv \mathbf{0}$ and heat moves by conduction only. The partition splits the cavity into two coupled half-enclosures, each developing its own convection cell; with increasing Ra the cells intensify, the wall and partition-face boundary layers thin, and the half-cavity cores stratify: the same boundary-layer transition as the single cavity (main text), now coupled across the conducting partition.

1656 10^6 , reductions of 3.7%, 18.2% and 24.0%. The reduction grows sharply with Rayleigh
 1657 number: at $Ra = 10^4$ transport is conduction-dominated and largely thickness-insensitive,
 1658 while at $Ra = 10^6$ convection carries most of the heat and the wider stagnant band
 1659 blocks proportionally more of it. This monotone, Rayleigh-amplified thickness sensitivity
 1660 reproduces the trend reported by Khatamifar et al. [5], who likewise find that Nu decreases
 1661 with partition thickness and that the effect is most pronounced at small k_r , where the
 1662 partition cannot compensate by conducting more efficiently.

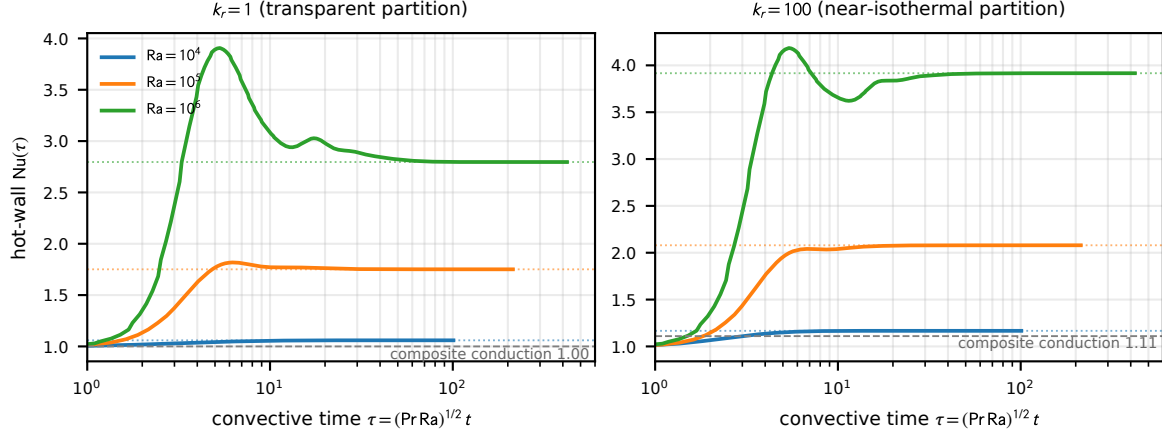
1663 **Transient development.** The benchmark is transient, and the RK3–CN integrator
 1664 resolves the full development from the quiescent conduction initial state (Supplementary
 1665 Figure S6a). The velocity is zero and the temperature is the linear profile $\theta = \frac{1}{2} - x$, the
 1666 conduction solution of the uniform cavity, so every case starts at $Nu = 1$. For $k_r = 1$
 1667 this is also the conduction equilibrium of the partitioned cavity. For $k_r = 100$ it is not.
 1668 The composite-conduction equilibrium of a fluid width $1 - T_p$ in series with a partition
 1669 of thickness T_p and conductivity ratio k_r is $Nu_{\text{cond}} = 1/[(1 - T_p) + T_p/k_r]$, which is 1.11
 1670 at $T_p = 0.1$, and the early rise of the $k_r = 100$ curves combines the diffusive adjustment
 1671 toward that value with the convective onset. Each case then passes through a convective
 1672 spin-up as the wall and partition-face boundary layers form and the half-cavity cells
 1673 establish (the steady flow structure is shown in Supplementary Figure S5), and relaxes to
 1674 the steady values reported in the main text. Our diffusion-time non-dimensionalization
 1675 t maps to Khatamifar’s convective time by $\tau = (\text{Pr} Ra)^{1/2} t$ (their reference velocity
 1676 $U_{\text{ref}} = (\alpha_f/H)\sqrt{\text{Pr} Ra}$); plotted against τ , the convective onset collapses across Rayleigh
 1677 number. For $k_r = 1$, where the start is the conduction equilibrium, all three curves leave

it within the first two convective units, so the spin-up follows the convective rather than the diffusive time scale. The overshoot above the steady value grows sharply with Ra (at $k_r = 1$ the hot-wall Nu peaks at 1.06, 1.82, and 3.91 for Ra = 10^4 , 10^5 , 10^6 , against steady values 1.06, 1.75, 2.79), the signature of the starting plume overshooting before the stratified core forms. For the near-isothermal partition ($k_r = 100$) the development carries a distinct secondary feature (a mild dip and recovery around $\tau \sim 8$) as the cold intrusion reaches the partition and the two half-cavities become thermally coupled across it, the conjugate counterpart of the boundary-layer impingement described by Khatamifar et al. [5]. The time to steady state increases with Ra in convective units ($\tau \approx 50$ to 220; equivalently the diffusion time t falls from ≈ 0.58 to 0.26), consistent with their reported development, largely complete by $\tau \sim 250$ over this range. We validate the transient pathway through this development structure (conduction start, Ra-collapsed convective spin-up, overshoot, and the conjugate coupling feature) together with the steady end-state, rather than a point-by-point overlay of $\text{Nu}(\tau)$, for which digitized reference data are not available.

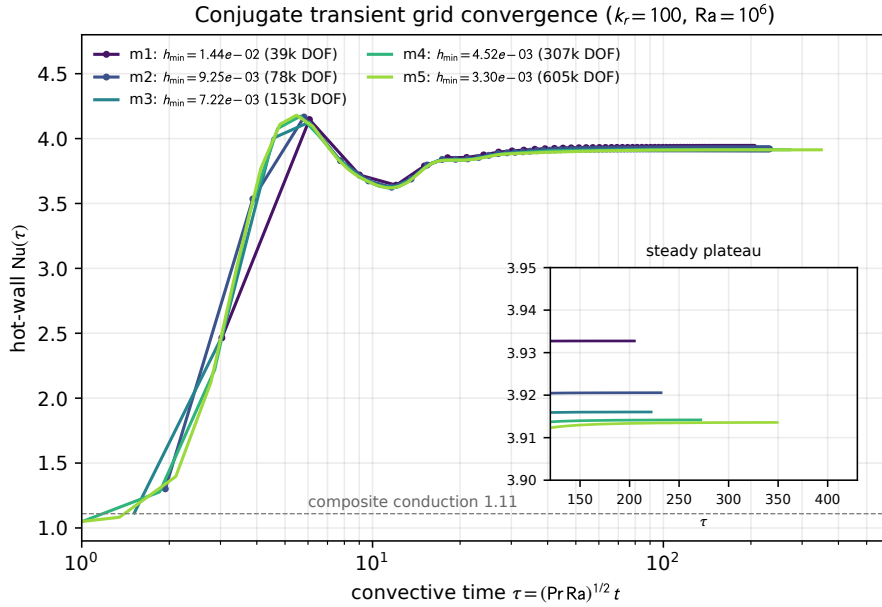
Supplementary Figure S6b shows that the transient development converges with the mesh, not only the steady endpoint. The $k_r = 100$, Ra = 10^6 history is the strongest-convection case in the validation set, where resolution is most demanding. Over the same five-mesh $\sqrt{2}$ refinement sequence the convective overshoot, the undershoot and the steady relaxation converge onto one curve. From the overshoot on ($\tau \geq 10$) the three finest meshes agree to within 0.3% at fixed τ (m3 against m5; 0.13% for m3 against m4 and 0.22% for m4 against m5). During the rise the logged samples differ by up to 6% at fixed τ , and part of that is sampling: every mesh logs Nu once per 200 steps, which is 3.0 convective units on m1, 1.5 on m3 and 0.7 on m5. The steady plateau converges monotonically from Nu = 3.933 (m1, $h_{\min} = 1.4 \times 10^{-2}$) to 3.913 (m5, $h_{\min} = 3.3 \times 10^{-3}$), and Richardson extrapolation of m3 to m5 gives 3.913, the main-text value. The plateau values of m3 to m5 span 0.06%.

Grid-convergence inputs behind Table 1. Supplementary Table S21 lists the per-mesh hot-wall Nusselt numbers behind the solution-verification rows of main-text Table 1, and the quantities derived from them; Table 1 quotes the RK3–CN entries. Nu is re-evaluated from the saved double-precision fields rather than read from the four-decimal solver log, which is too coarse to difference across a $\sqrt{2}$ sequence. At Ra = 10^5 the successive differences are of the order of the steady-state tolerance, so the observed order is tolerance-limited and only the GCI is quoted. At Ra = 10^4 the successive differences do not shrink, so no order is fit and Table 1 quotes the five-mesh spread instead.

Same-scheme throughput and time to a converged Nu. Supplementary Table S22 collects the six Tier B conjugate runs plotted in main-text Figure 3d,e, both per step and



(a) **Transient development of the hot-wall Nusselt number** (GNN, $T_p = 0.1$, mesh $h_{\min} = 7.2 \times 10^{-3}$). $Nu(\tau)$ from the quiescent conduction start; left $k_r = 1$, right $k_r = 100$. Dashed line: each panel's composite-conduction equilibrium, 1.00 for $k_r = 1$ and 1.11 for $k_r = 100$. Dotted lines: the steady values reported in the main text. The development is described in the text above.



(b) **Grid convergence of the transient conjugate development** (GNN, conjugate cavity, $k_r = 100$, $Ra = 10^6$). Hot-wall $Nu(\tau)$ over the five-mesh $\sqrt{2}$ refinement sequence. Dashed line: the composite-conduction equilibrium $Nu_{\text{cond}} = 1.11$; all runs start from the linear profile at $Nu = 1$, and the $t = 0$ sample is omitted from the logarithmic axis. The coarser curves are sparsely sampled through the rise, so their samples are drawn as markers. Inset: the steady plateau. Convergence is quantified in the text above.

Supplementary Figure S6. Transient conjugate development and its grid convergence. Both panels show the hot-wall Nusselt number against Khatamifar's convective time $\tau = (Pr Ra)^{1/2} t$ for the conjugate cavity, first across regimes and then under mesh refinement of the same transient.

1715 end to end. The two implementations differ only in the linear solver and the hard-
 1716 ware: graph multigrid on one H200 against PETSc CG + GAMG on 32 CPU cores, same
 1717 rotational-IPCS RK3–CN scheme, P_2 – P_1 mesh, fixed time step and 10^{-6} pressure toler-
 1718 ance (Section 4.2). The step count to a steady Nu is set by the shared scheme and time

Supplementary Table S21. Per-mesh conjugate Nusselt numbers and the grid-convergence inputs behind Table 1. Hot-wall Nu of the conjugate cavity ($k_r = 1$, $T_p = 0.1$) on the five-mesh $\sqrt{2}$ sequence (m1 to m5, characteristic length l_c in parentheses), for the RK3–CN and ARS(2,2,2) integrators, re-evaluated from the saved double-precision fields. Observed order p , Richardson value Nu_∞ and GCI (safety factor 1.25, $r = \sqrt{2}$) use m3, m4 and m5. Where p cannot be fit because the successive differences do not shrink, the GCI is given under assumed orders 2 and 3 and the observed-order entries are left blank; Table 1 quotes the m1–m5 spread there. The last row is the relative difference between the two integrators, maximised over the five meshes.

Mesh (l_c)	$\text{Ra} = 10^4$	$\text{Ra} = 10^5$	$\text{Ra} = 10^6$
<i>RK3–CN</i>			
m1 (0.020)	1.059 482 8	1.752 432 1	2.807 482 6
m2 (0.0141)	1.059 421 1	1.751 346 8	2.799 184 1
m3 (0.010)	1.059 375 7	1.750 969 6	2.796 042 1
m4 (0.00707)	1.059 274 1	1.750 837 5	2.794 810 4
m5 (0.005)	1.059 148 4	1.750 800 7	2.794 362 1
observed order p (m3–m5)	–	3.68	2.92
Nu_∞ (Richardson)	–	1.750 786 5	2.794 105 5
GCI at m5, observed p (%)	–	0.0010	0.0115
GCI at m5, assumed $p = 2$ (%)	0.0148	0.0026	0.0201
GCI at m5, assumed $p = 3$ (%)	0.0081	0.0014	0.0110
spread m1–m5 (%)	0.032	0.093	0.470
<i>ARS(2,2,2)</i>			
m1 (0.020)	1.059 488 6	1.752 431 8	2.807 482 6
m2 (0.0141)	1.059 447 2	1.751 343 2	2.799 184 0
m3 (0.010)	1.059 433 2	1.750 960 2	2.796 042 1
m4 (0.00707)	1.059 427 7	1.750 815 0	2.794 820 4
m5 (0.005)	1.059 416 7	1.750 763 9	2.794 370 6
observed order p (m3–m5)	–	3.01	2.88
Nu_∞ (Richardson)	–	1.750 736 1	2.794 108 5
GCI at m5, observed p (%)	–	0.0020	0.0117
GCI at m5, assumed $p = 2$ (%)	0.0013	0.0037	0.0201
GCI at m5, assumed $p = 3$ (%)	0.0007	0.0020	0.0110
spread m1–m5 (%)	0.007	0.095	0.469
scheme difference, max over m1–m5	2.5×10^{-4}	2.1×10^{-5}	3.6×10^{-6}

step, so it is the same for both. The GPU run time is measured. The CPU run time is projected: the measured GPU run time multiplied by the CPU/GPU per-step ratio of the stationary late window. That projection assumes the late-window ratio holds over the whole trajectory, which was not measured. The late-window GPU cost per step is also higher than the whole-run average (at m1, 40.5 ms against $7.2 \text{ min}/15380 = 28.1 \text{ ms}$), so the run time is not the step count times the late-window cost; the ratio, not the absolute per-step cost, is what the projection carries over.

Supplementary Table S22. Same-scheme conjugate throughput: per step and to a converged Nu ($k_r = 1$, $T_p = 0.1$, $Ra = 10^6$; graph multigrid on one H200 against PETSc CG + GAMG on 32 CPU cores). Per-step costs are measured on both sides in each implementation’s stationary late window. Steps is the count to the shared steady-state criterion ($|\Delta Nu/Nu| < 10^{-5}$). The speedup is the per-step ratio and, by construction, the run-time ratio.

Mesh (total DOF)	per step, measured (ms)		Steps	to converged Nu (min)		Speedup
	GPU	CPU		GPU, measured	CPU, projected	
m1 (3.9×10^4)	40.50	50.71	15 380	7.2	9.1	1.25
m2 (7.8×10^4)	41.25	90.00	23 922	11.4	24.9	2.18
m3 (1.5×10^5)	42.00	161.43	29 408	14.6	56.1	3.84
m4 (3.1×10^5)	44.00	304.29	46 821	24.0	165.8	6.92
m5 (6.0×10^5)	47.00	538.57	71 663	40.2	460.8	11.46
m6 (1.2×10^6)	60.75	1452.86	136 809	108.2	2588.0	23.92

S7 An applied example: conjugate heat transfer in an engine cylinder

The conjugate cavity benchmark of Section 2.5 validated the conjugate solver against a published numerical reference; this last example takes the *same* solver to an applied, three-dimensional geometry: the conjugate heat transfer of a single-cylinder internal-combustion engine, the configuration of the ANSYS Forte tutorial [7]. A cylindrical gas chamber, the bore, is fully enclosed in a metal block, the head and liner. The combustion load enters as a prescribed, cycle-averaged wall heat flux q'' per unit area on the chamber wall, which is the fluid–solid interface itself. The interface therefore carries a source: the flux jump $\mathbf{q}_f \cdot \mathbf{n}_f + \mathbf{q}_s \cdot \mathbf{n}_s$ there is $-q''$ rather than zero, with $q'' > 0$ a heat input (Equation (6) in Methods). Temperature is continuous across it, so the source is deposited on the shared interface nodes and conducted into both the metal and the gas rather than assigned to one side. A coolant jacket holds the outer metal faces at a fixed temperature. The in-cylinder gas convects under buoyancy and the metal conducts the wall heat to the coolant. The quantity of interest is the steady metal temperature, the fire-deck (head) surface and the liner, which sets thermal-fatigue and knock margins in design practice.

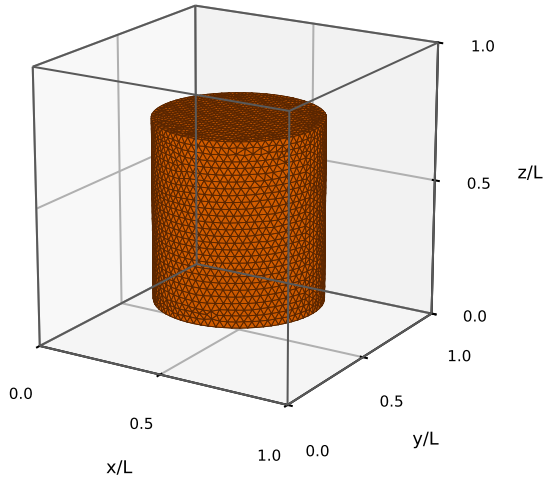
One monolithic system vs. a partitioned coupling. The reference workflow is *partitioned*: Forte solves the transient reacting gas, ANSYS Fluent solves the steady solid, and System Coupling exchanges fields across the interface (Forte sends the cycle-averaged wall heat flux, Fluent returns the wall temperature), iterating a Dirichlet–Neumann loop with quasi-Newton under-relaxation to convergence at each coupling step. We instead solve *one* system: the conductivity-weighted energy equation over gas and metal together (Section 2.5), the gas momentum, and the graph-multigrid pressure projection, with the fluid–solid interface heat balance, including the prescribed source, enforced *weakly* through the shared C^0 Galerkin discretization. There is no outer coupling loop and

no interface under-relaxation, so the coupling carries no lagged partitioning error. The pressure solve is the same graph-multigrid preconditioner used throughout this paper, here exercised by the buoyant in-bore convection (≈ 16 – 23 iterations per solve). The engine case is solved by exactly the method of Sections 2.7 and 2.5; only the geometry and the thermal load are new.

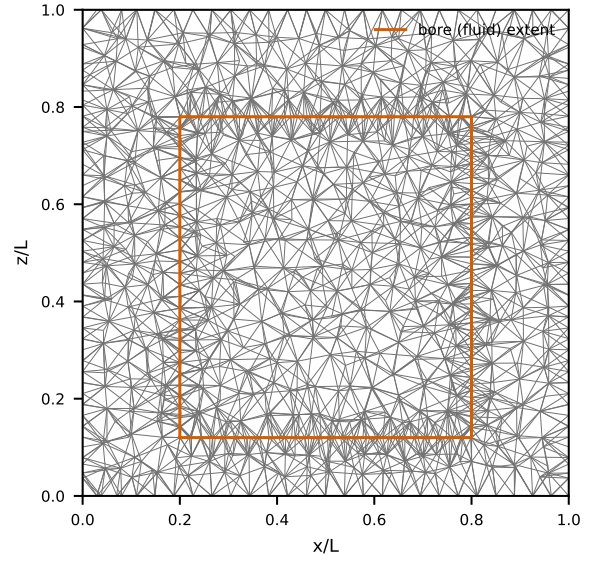
Geometry, mesh, and boundary conditions. The geometry (Supplementary Figure S7a) is a cylindrical gas bore of radius $R = 0.3L$, centred at $(x, y) = (0.5, 0.5)L$ and extending from $z = 0.12L$ (floor) to $z = 0.78L$ (fire deck), fully enclosed in a cubic metal block of side L ; the physical specification sets $L = 0.1$ m. The boundary conditions are the prescribed cycle-averaged heat flux q'' on the bore chamber wall (the fluid–solid interface), a fixed coolant temperature on the six outer block faces (Dirichlet), and no-slip for the gas on the chamber wall. The double prime on q'' follows the heat transfer convention for a quantity taken per unit area, with units W m^{-2} ; it does not denote differentiation. The mesh is a conforming body-fitted tetrahedralization (`gmsh occ.fragment`), so the fluid–solid interface is resolved by element facets exactly (Supplementary Figure S7b), the piecewise conductivity is element-wise constant and the prescribed flux is a facet integral. Supplementary Table S23 quantifies the three quantity-of-interest meshes: the minimum dihedral angle is $\approx 12^\circ$ with *no* slivers below 10° , the geometry is $\approx 75\%$ solid by cell count ($\approx 81\%$ by volume) at every resolution, and the interface facet size falls from $0.12R$ to $0.07R$, about 14 elements across the bore radius on the finest mesh.

Nondimensionalization and scope. The metal-to-gas conductivity ratio is $k_s/k_f = k_r = 10^3$, representative of metal over a combustion gas, and the problem is nondimensionalized on the conduction temperature scale $\Delta T_{\text{ref}} = q''L/k_s$, which fixes the buoyancy Rayleigh number at $\text{Ra} = 10^5$ and makes the nondimensional wall flux $q^* = k_s/k_f$. Because ΔT_{ref} itself enters the Rayleigh number, the groups Ra and k_s/k_f and the dimensional flux, length and conductivity are not all independently specifiable for one fixed real gas; the fluid properties are whatever makes $\text{Ra} = 10^5$ consistent with the chosen scale. The case is therefore a nondimensional similarity problem. Before any engine run we verify the new assembly on a two-material conduction slab with a prescribed interface flux. The computed interface temperature reproduces the analytic value $q^*/[2(k_s + k_f)]$ to machine precision, with relative error $< 10^{-13}$ for $k_s/k_f \in \{1, 10^2, 2 \times 10^3\}$, because the P_2 elements represent the piecewise-linear field exactly. That slab check and the conjugate manufactured-solution study (Supplementary Note S1.3, which spans $k_s/k_f \in \{10^2, 10^3\}$; Supplementary Table S4) are the verification of the conjugate assembly. The forward solve is then run without calibration to a target temperature, from a physical specification: a cast-iron head/liner ($k_s = 55 \text{ W m}^{-1}\text{K}^{-1}$), coolant at 360 K, cycle-averaged flux $q'' = 0.5 \text{ MW m}^{-2}$, block scale $L = 0.1$ m. The resulting metal temperature is *reported*,

(a) geometry: bore chamber wall in metal block



(b) tetrahedral mesh, mid-plane $y = 0.5$



Supplementary Figure S7. Engine conjugate geometry and mesh. (a) The cylindrical gas bore (chamber wall, triangulated) enclosed in the cubic metal block (wireframe); nondimensional coordinates on the unit block. (b) The body-fitted tetrahedral mesh on the mid-plane $y = 0.5$ (medium mesh, Supplementary Table S23), with the bore (fluid) extent outlined. The fluid–solid interface, the flux-loaded chamber wall, is aligned to element facets.

not tuned. The Kelvin figures below are an illustrative dimensional mapping of the similarity solution, not a specific-gas prediction. No validation against engine physics was performed, and agreement with operating cylinder-head temperatures is a plausibility check, not an independent quantitative test. The combustion load is idealized as a prescribed, cycle-averaged wall heat flux and the coolant as a fixed-temperature boundary, so radiation, valve-driven and compressible in-cylinder gas motion, combustion chemistry, cycle-to-cycle variation, and the detailed coolant-jacket flow are all absent. The reacting-flow Forte/Fluent tutorial is a partitioned-workflow reference for the coupling structure only, *not* an equivalent simulation and *not* a validation target. The dimensionless heat sink of Supplementary Note S3.5 remains the cleaner quantitative application; this case demonstrates the same solver on an applied geometry.

Result. Extended Data Fig. 5 shows the steady mid-plane temperature. The gas chamber is heated through the flux-loaded wall, buoyancy lifts the hot gas to the fire deck, and the metal carries a smooth gradient from the hot deck to the cooled outer faces. The solver-determined output is the *nondimensional* conjugate field. The fire-deck surface temperature is $\theta = 0.1833$, grid-independent to four significant figures across the three meshes (Supplementary Table S23). One GPU-native monolithic solver, the same method used for the canonical benchmarks, therefore produces a grid-converged conjugate field on a realistic body-fitted engine geometry with no partitioned outer iteration. The volume-averaged liner-side metal is slower to settle. It drifts monotonically by ≈ 3 K

between the two finest meshes and has not reached a grid plateau, so only the flux-loaded fire-deck peak is used as a quantitative output and the liner-side mean is indicative.

Mapping θ to Kelvin uses $T = T_{\text{cool}} + \theta q'' L / k_s$. At fixed nondimensional groups the flux q'' , the length L and the coolant temperature enter only through that closed-form rescaling, so they leave θ , and hence the grid behaviour, unchanged. The solid conductivity k_s enters both the scale and, through the ratio k_s/k_f , the field itself, so it is not a free rescaling. For the cast-iron specification above ($q'' = 0.5 \text{ MW m}^{-2}$, $L = 0.1 \text{ m}$, $k_s = 55$, coolant 360 K) the grid-converged fire deck maps to $\approx 527 \text{ K}$, the order expected for an operating cast-iron cylinder head [8], and the liner-side mean to $\approx 401 \text{ K}$. The implied rise from coolant to fire deck is $\approx 167 \text{ K}$, which is large for the constant-property Boussinesq model used here, so a dimensional prediction would also require justifying that model at such a temperature difference.

Supplementary Table S23. Engine meshes and metal-temperature QoI, grid convergence. Forward solve without calibration to a target temperature ($\text{Ra} = 10^5$, $k_s/k_f = 10^3$, $q'' = 0.5 \text{ MW m}^{-2}$, $L = 0.1 \text{ m}$, coolant 360 K), mapped to Kelvin for a cast-iron head/liner ($k_s = 55$) as an illustrative dimensional mapping. “min dih.” is the minimum tetrahedron dihedral angle, “iface facets” the number of fluid–solid interface triangles and h_Γ/R their mean edge length relative to the bore radius. The fire-deck surface (nondimensional $\theta = 0.1833$) is grid-independent.

Mesh	tets	P ₂ DOF	min dih. (deg)	iface facets	h_Γ/R	fire-deck (K)	liner-side mean (K)
coarse	27573	41036	13.7	3178	0.121	526.6	394.5
medium	61327	88897	12.0	5684	0.091	526.6	397.6
fine	125781	178636	12.0	9260	0.071	526.6	400.6

Throughput on the body-fitted geometry. Every other cross-solver throughput comparison in this work is set on a box. The engine case tests throughput on a curved, body-fitted fluid–solid interface. Only the FEniCSx reference is compared. The structured convolutional solver is Cartesian by construction and would have to staircase the interface, which would confound geometry with discretization. OpenFOAM supports body-fitted conjugate meshes, but the configuration used in the cavity benchmark was not extended to this case; a body-fitted OpenFOAM baseline would be a useful comparison, though still Tier C because the discretization differs. FEniCSx shares the P₂–P₁ discretization exactly, so it is the only controlled comparison available on this mesh.

The protocol is the conjugate-cavity protocol of Section 2.5: an exclusive node, the CPU run on 32 cores, both implementations driving the pressure to the same 10^{-6} , identical mesh, time step, step count and rotational-IPCS update, three repetitions with the median reported, and only the linear-algebra backend and the execution device changed. This is Tier B (Methods). Supplementary Table S24 gives the result. The pattern matches the conjugate cavity: the GPU per-step cost is nearly flat across a $21\times$ size range while

the CPU sparse solve grows with the problem, so the ratio crosses over with size. At the coarsest mesh the CPU is ahead, at $0.81\times$; at the finest the GPU is $23.9\times$ faster. That the GPU loses at 3.6×10^4 DOF is the same small-problem loss reported for the cavity, not a geometry effect. Both implementations return the same physics: the fire-deck peak agrees to 0.22% or better at every mesh, which is what makes the timing a comparison of two ways to take the same step. The mass and diffusion solves are the bulk of the CPU step, so their tolerance is matched at 10^{-10} in both implementations; leaving them mismatched at 10^{-7} on the GPU inflates the ratio by 1.25–1.28 $\times$. Matching both implementations at 10^{-7} instead gives 0.84, 11.61 and 24.01, within 0.5% of the reported ratios at the two finer meshes and 3.7% at the coarsest, so the result does not turn on which matched tolerance is chosen. Doubling the CPU allocation from 16 to 32 cores moved the CPU time by -1 to $+3\%$ at every mesh, so PETSc CG + GAMG does not thread-scale here and the ratios are not an under-resourced baseline. One protocol difference should be noted. The conjugate-cavity GPU run uses CUDA-graph capture, worth roughly $2\times$ on pressure-dominated work. The engine GPU driver runs eagerly; CUDA-graph capture was not evaluated for this case.

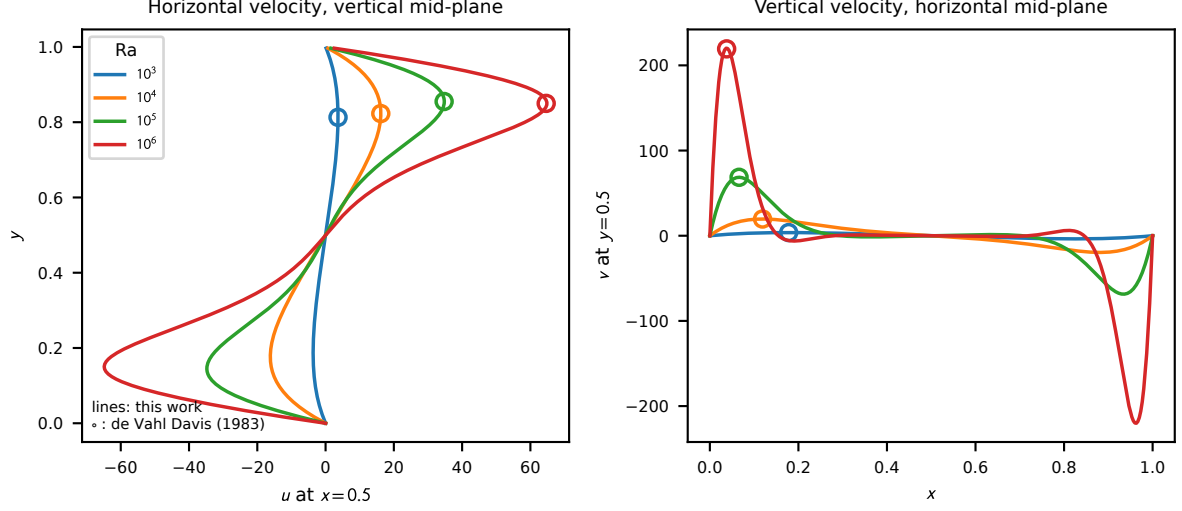
Supplementary Table S24. Same-discretization throughput on the body-fitted engine geometry (Tier B, Section 4.2). Graph-multigrid on one GPU against PETSc CG + GAMG on 32 CPU cores, protocol as in the text. The right-hand columns give the fire-deck peak θ from each implementation, confirming both solve the same problem. These are fixed step-count timing runs rather than the converged steady solutions of Supplementary Table S23, so their θ agrees with the converged 0.1833 to three significant figures rather than four.

P ₂ DOF	coupled DOF	GPU (ms/step)	CPU (ms/step)	ratio	θ GPU	θ CPU
8568	35542	161.8	130.9	0.81	0.1830	0.1829
88897	367586	226.0	2612.5	11.56	0.1831	0.1830
178636	738087	241.6	5771.0	23.89	0.1831	0.1835

Distributed scaling. The engine solver distributes with the same x -slab decomposition, graph-multigrid pressure and two-level coarse correction as the heat sink. Its strong scaling on the ceiling mesh and its weak scaling to $\approx 4.9 \times 10^8$ DOF on eight GPUs are reported with the other distributed cases in Supplementary Note S3.6 (Supplementary Tables S10 and S13).

S8 Full benchmark tables and cavity flow reference

The three-dimensional Nusselt table, the wall-clock table, the per-SIMPLE-iteration cost table, the 2D Nusselt validation against published benchmarks, the high-Rayleigh ($Ra = 10^8$) robustness comparison and the cavity centreline-profile figure are here. The



Supplementary Figure S8. Centreline velocity profiles against the de Vahl Davis (1983) benchmark. Horizontal velocity u on the vertical mid-plane $x = 0.5$ (left) and vertical velocity v on the horizontal mid-plane $y = 0.5$ (right), in the κ/L velocity scaling of the benchmark. Solid lines: this work (GNN-SIMPLE, h_{128}); open circles: the de Vahl Davis reference extrema. The peak magnitudes and locations agree to within 1.5% at every Rayleigh number. The near-wall jets sharpen with increasing Ra , which tracks the boundary-layer transition.

2D Nusselt table is Extended Data Table 1. The main text keeps the work-precision comparison.

Supplementary Table S25. Converged Nu on the 3D Boussinesq cavity benchmark. All entries are converged through the shared stopping test. The FEniCSx h_{64} reference runs are the costliest (a single P_2 - P_1 outer iteration costs ≈ 44 s and convergence needs 3-5 thousand; Supplementary Table S27); the $Ra = 10^4$, h_{64} run uses $\alpha_U = 0.5$ (the value the Ra-adaptive rule already applies at $Ra \geq 10^5$) because at $\alpha_U = 0.7$ that marginal cell stalls on the conduction branch. The reference column is the pseudo-spectral average Nu of Tric et al. [9].

Ra	mesh	CNN	GNN	FEniCSx	OF12	Tric (2000)
10^3	h_{16}	1.078	1.072	1.072	1.076	
	h_{32}	1.073	1.071	1.071	1.072	1.0700
	h_{64}	1.071	1.071	1.071	1.071	
10^4	h_{16}	2.152	2.072	2.072	2.129	
	h_{32}	2.077	2.058	2.058	2.075	2.0542
	h_{64}	2.060	2.053	2.056	2.060	
10^5	h_{16}	5.478	4.504	4.504	4.826	
	h_{32}	4.512	4.375	4.375	4.480	4.3371
	h_{64}	4.377	4.342	4.343	4.373	

High-Rayleigh robustness at $Ra = 10^8$. On the 2D Rayleigh-Bénard cell at h_{128} , from the same initial condition, our fully explicit finite difference CNN configuration is throttled to its advective CFL step $\Delta t \approx 9.3 \times 10^{-8}$ and diverges to a non-finite state at $t \approx 0.040$, before the flow develops. The semi-implicit RK3-CN scheme (implicit diffusion),

Supplementary Table S26. Per-SIMPLE-iteration cost (ms), 2D and 3D benchmarks. Cost of one SIMPLE outer iteration, measured in isolation on an exclusive node. All four solvers advance one SIMPLE outer iteration per step: OF12 is the steady-state SIMPLE cavity (`foamRun -solver fluid, ddtSchemes steadyState`, exclusive node), the same steady-SIMPLE run that also provides its wall-clock time (main text); OpenFOAM’s native transient PIMPLE solver is not the apples-to-apples target for this steady benchmark. The four discretizations differ (CNN structured finite difference, GNN/FEniCSx P_2 – P_1 FEM, OF12 cell-centred finite volume method (FVM)), so this is a cross-discretization comparison at a shared iteration unit, not a same-method ratio. GNN = laplacian-preconditioned multigrid PCG with CUDA-graph capture; FEniCSx and OF12 on 32 CPU cores.

Ra	mesh	CNN	GNN	FEniCSx	OF12
<i>Two-dimensional benchmark</i>					
10^4	h_{64}	157.4	11.6	80.5	3.8
	h_{128}	138.9	12.1	304.6	17.4
	h_{256}	163.6	32.5	1160	78.2
10^5	h_{64}	173.4	11.9	80.0	4.0
	h_{128}	183.6	12.8	306.9	15.9
	h_{256}	168.8	17.6	1182	75.3
10^6	h_{64}	174.7	13.0	80.0	4.3
	h_{128}	183.1	14.0	310.0	16.6
	h_{256}	168.6	17.6	1184	70.9
<i>Three-dimensional benchmark</i>					
10^3	h_{16}	408.1	22.7	578.5	4.7
	h_{32}	461.5	31.7	5228	39.0
	h_{64}	458.2	164.6	44 084	444.3
10^4	h_{16}	418.9	19.1	604.6	4.8
	h_{32}	463.6	32.1	5184	38.3
	h_{64}	498.7	164.6	44 698	428.3
10^5	h_{16}	434.6	19.8	594.6	5.3
	h_{32}	462.7	32.8	5281	38.0
	h_{64}	499.2	167.9	44 476	446.8

run here on the GNN and also used by the CPU FEM implementation, integrates the same case stably and reaches $\langle \text{Nu} \rangle \approx 25.6$ by $t = 0.3$. This demonstrates the stability margin of implicit diffusion for *this* configuration, not that explicit structured methods fail at $\text{Ra} = 10^8$ in general.

S9 Single-GPU throughput at scale

The main-text comparisons are at moderate resolution. To characterize behaviour at the single-device limit we push one H200 (141 GB) toward its memory ceiling along two axes: the pressure solve in isolation, and the full per-step cost of the structured and unstructured solvers. Both are single-GPU, single-operator characterizations. The whole-step cost of the same pressure backends, at the converged-benchmark sizes, is in Supplementary Note S2. That comparison dilutes the per-solve ranking reported below, and on the cavity reverses it.

Supplementary Table S27. Wall-clock time to convergence (minutes), matched-mesh end-to-end times. GPU solvers (CNN, GNN) on one H200; FEniCSx (P₂-P₁, plain SIMPLE) and OpenFOAM v12 (cell-centred FVM, steady-SIMPLE mode) on 32 CPU cores. Reported for orientation, *not* like-for-like (Tier C, Section 4.2). Rows match by *mesh label*, not problem size, so these are times to a converged Nu at a common target rather than costs per degree of freedom; Section S9.2 gives the count per mesh and discretization family. Per-iteration cost is in Supplementary Table S26 and per-mesh accuracy in Extended Data Table 1 and Supplementary Table S25. *Two-dimensional block, directly measured.* Each entry is setup + solve; setup ($\lesssim 7$ s for the GPU and FEniCSx solvers, 6–14 s for OF12 mesh generation) is negligible except at the sub-minute OF12 cells. *Three-dimensional block, reconstructed* as (iterations to convergence) $\times$ (isolated per-iteration cost, Supplementary Table S26) rather than directly timed. The two FEniCSx h_{64} , $Ra \geq 10^4$ entries are directly measured and agree with the reconstruction to 0.1%, which validates the reconstructed cells. At $Ra = 10^3$ the flow is nearly conduction-dominated, so that row reflects the ease of the problem rather than solver capability: OpenFOAM leads by $\approx 7\%$ there, while the graph solver leads by $1.6\text{--}3.4\times$ at the two convective Rayleigh numbers. FEniCSx counts are plain SIMPLE except $\ddagger$, the h_{64} , $Ra = 10^3$ cell, which is Anderson-accelerated; its plain-SIMPLE wall is several times larger, so the tabulated value is a conservative lower bound.

Ra	mesh	CNN	GNN	FEniCSx	OF12
<i>Two-dimensional benchmark (directly measured)</i>					
10^4	h_{64}	9.1	1.5	16.9	0.5
	h_{128}	25.2	3.3	123.4	3.9
	h_{256}	70.5	42.6	838.4	57.2
10^5	h_{64}	10.3	0.7	16.4	0.6
	h_{128}	34.3	3.9	135.3	3.6
	h_{256}	99.2	24.4	854.5	51.9
10^6	h_{64}	18.9	0.9	19.2	0.6
	h_{128}	38.5	3.1	142.4	3.3
	h_{256}	131.9	12.9	876.6	44.8
<i>Three-dimensional benchmark (reconstructed)</i>					
10^3	h_{16}	6.1	0.5	6.7	0.2
	h_{32}	16.9	2.1	130.7	1.8
	h_{64}	55.2	48.6	808.2	$\ddagger$ 45.6
10^4	h_{16}	4.9	0.2	6.0	0.2
	h_{32}	12.4	1.6	112.3	2.3
	h_{64}	41.0	11.9	3680.5	40.6
10^5	h_{16}	7.2	0.3	7.9	0.3
	h_{32}	13.1	1.0	114.4	2.2
	h_{64}	36.7	23.0	2350.4	35.8

S9.1 Isolated pressure solve

Extended Data Fig. 4a reports the assembled frozen-Laplacian pressure solve (a single frozen operator and right-hand side, the RBC/CHT pressure path) on one H200, comparing the fp64 graph-multigrid PCG against two GPU alternatives: a mixed-precision variant (fp64 outer Krylov, fp32 V-cycle preconditioner, otherwise bit-identical) and NVIDIA AmgX (classical algebraic multigrid, fp64), all solved to the same 10^{-6} relative tolerance. Both alternatives accelerate the solve and the gain grows with size: at 1.68×10^7 pres-

Supplementary Table S28. Validation of the 2D converged Nusselt number against established benchmarks. Differentially-heated square cavity, air ($\text{Pr} = 0.71$). “This work” is the finest-converged-mesh (h_{256}) GNN–SIMPLE value (the $\text{Ra} \geq 10^4$ rows are those of Extended Data Table 1; the $\text{Ra} = 10^3$ entry is from the same h_{256} sweep); the references are the benchmark mean wall Nusselt number of de Vahl Davis [13] and, at $\text{Ra} = 10^6$, the spectral solution of Le Quéré [14]. Agreement is within 0.3% throughout (within 0.005% at $\text{Ra} = 10^6$).

Ra	This work (GNN)	de Vahl Davis (1983)	Le Quéré (1991)
10^3	1.117	1.118	–
10^4	2.245	2.243	–
10^5	4.519	4.519	–
10^6	8.8257	8.800	8.8252

sure DOF the mixed-precision solve is $1.29\times$ and AmgX $1.39\times$ faster *per solve* than fp64 multigrid. In peak device memory AmgX is the leanest ($2.15\times$ below fp64 at scale), while the mixed-precision path costs $\sim 44\%$ more (the fp32 hierarchy is stored alongside the fp64 operator).

Three limitations bound it. (i) *Baseline*: the multigrid here runs to convergence (≈ 50 CG iterations), matching the RBC configuration but not the cycle-capped cavity/CHT ones. (ii) AmgX is run in a lean integration, with no per-solve residual recompute and no duplicate operator copy, so its time and memory are not adapter-confounded. Its execution mode also differs from the CUDA-graph captured multigrid baseline (Supplementary Note S2), so part of the growth of the AmgX time advantage with size is the capture advantage of the baseline decaying as kernels enlarge. The memory comparison is unaffected. (iii) We do not extrapolate beyond 1.68×10^7 DOF, and a head-to-head against *distributed* production AMG remains future work (Section 2.7).

S9.2 Per-step cost and capacity of the structured and unstructured solvers

Extended Data Fig. 4b plots the full per-step wall time against total DOF (all fields advanced per step) as each solver is taken to the single-GPU limit. On a matched total-DOF basis the structured CNN step is $\approx 3.5\text{--}4.5\times$ cheaper than the unstructured GNN step, a gap that reflects the methods’ different discretizations and integrators, not the linear solver: the CNN step is a complete two-stage explicit Heun advance with one pressure Poisson solve, and the GNN step a complete three-substage implicit RK3–CN advance with a pressure projection at every substage. Per step means one advance of the solution by one time step in both solvers, and neither number is per stage. It is therefore not a like-for-like step (cf. Table S30). The capacity difference is larger: the matrix-free CNN reaches 1.3×10^8 total DOF on one H200, whereas the GNN’s stored $\text{P}_2\text{--}\text{P}_1$ finite element operators bound its full solver near 5.5×10^7 total DOF on this

problem (consistent with the capacity result of Supplementary Table S10), beyond which it exhausts device memory. Bytes per degree of freedom differ substantially across the problems in this paper, because the stored operator set and the number of fields differ, so the per-problem ceilings quoted in Supplementary Note S3 are not comparable to one another and are stated with their measured peak memory where available. The CNN’s geometric multigrid requires power-of-two grids, so its samples sit at h_{512} – h_{4096} ; the comparison is read across the overlapping range.

For reference, the total degrees of freedom of the cavity benchmark (all fields $u, v, (w,) p, T$) are Ra-independent. Colocated (CNN, OF12) against P₂–P₁ FEM (GNN, FEniCSx): in 2D 16384 against 62780 at h_{64} , 65536 against 248575 at h_{128} , and 262144 against 989664 at h_{256} ; in 3D 20480 against 121576 at h_{16} , 163840 against 873114 at h_{32} , and 1310720 against 6614191 at h_{64} . At a fixed mesh the FEM pair therefore carries $\approx 3.8\times$ (2D) and $5\text{--}6\times$ (3D) more unknowns than the colocated solvers, so a fixed-mesh comparison favours the colocated solvers on raw count, and the GNN per-iteration costs in Supplementary Table S26 are obtained on the larger FEM system.

S10 Statistical convergence of the turbulent Rayleigh–Bénard Nusselt number

The turbulent Rayleigh–Bénard time-averaged Nusselt number $\langle \text{Nu} \rangle$ used in the main-text throughput comparison (Section 2.4) is a statistic of a chaotic flow. We assess temporal convergence using a long trajectory, warm-up and averaging-window sensitivity, and block-based uncertainty estimates. We integrate the $\Gamma = 2$, $\text{Ra} = 10^7$, $\text{Pr} = 0.71$ cell on the h_{128} mesh for 1.5 thermal-diffusion times, equivalently $\approx 4.0 \times 10^3$ convective turnover (free-fall) times, since one diffusion time equals $\sqrt{\text{PrRa}} \approx 2665$ turnovers, logging the instantaneous hot- and cold-plate Nusselt numbers every ten steps (5.9×10^4 samples). The plate-averaged instantaneous $\text{Nu}(t)$, its running mean, autocorrelation, and blocking-curve convergence are shown in Supplementary Figure S9.

Time average and confidence interval. Discarding a 0.15-diffusion-time (≈ 400 -turnover) warm-up, the average over the remaining $\approx 3.6 \times 10^3$ turnovers is $\langle \text{Nu} \rangle = 14.04$. The Nusselt autocorrelation (Supplementary Figure S9b) resolves two time scales: a fast decorrelation at $\tau_{\text{dec}} \approx 0.9$ turnovers (first zero crossing) and a coherent large-scale-circulation oscillation of period ≈ 4.5 turnovers, of which the averaging window spans ~ 800 periods. Because that oscillation makes the naive integrated autocorrelation time an unreliable variance estimator, we quote a 95% confidence interval from the Flyvbjerg–Petersen blocking method [10]. We quote the *largest* interval over all statistically reliable block lengths (those leaving at least 50 blocks), ± 0.025 , which is attained at a block length

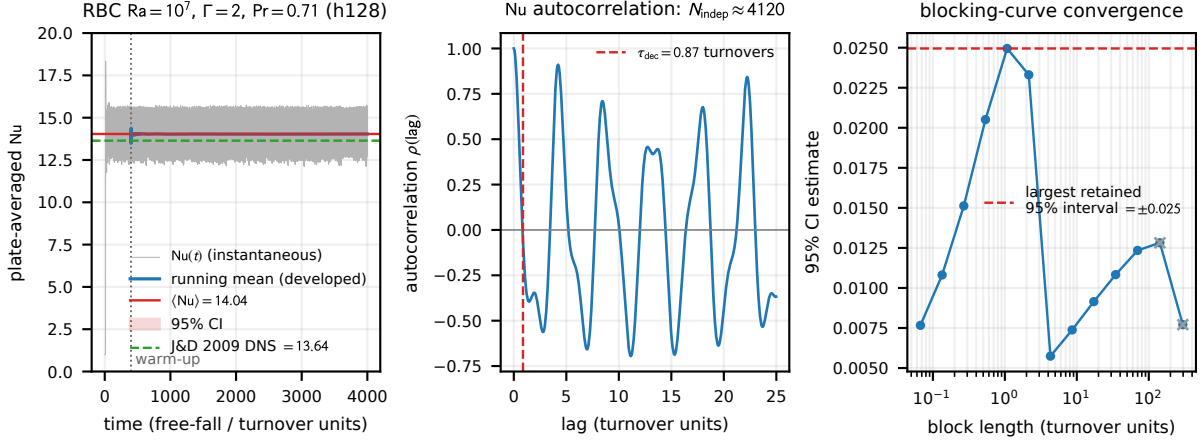
of ≈ 1.1 turnovers, just above the decorrelation time; coarser blockings return smaller intervals, so this is a conservative bound rather than a plateau value (Supplementary Figure S9c), giving

$$\langle \text{Nu} \rangle = 14.04 \pm 0.025 \quad (95\% \text{ confidence}), \quad (\text{S4})$$

+2.9% above the Johnston–Doering two-dimensional direct-numerical-simulation (DNS) correlation $0.138 \text{ Ra}^{0.285} = 13.64$ [11]. This long integration establishes the *temporal* (statistical) convergence of the time average, the property the throughput comparison relies on. The residual offset from the DNS correlation is therefore *not* a temporal-sampling artefact: the ± 0.025 interval, which is the single-trajectory temporal interval, is two orders of magnitude narrower than the offset itself. The remaining candidates are discretization, realization-to-realization scatter (quantified below, and of order 1%) and the correlation’s own spread. The grid-refinement study below (Supplementary Table S29) places both solvers near 14.0, about 3% above the correlation and approaching it from opposite sides; we caution that the value quoted here comes from the long-time average (a 1.35-diffusion-time window) and is not directly comparable with the shorter fixed-window ensemble used for the throughput tables. For scale, the offset is comparable to the reported few-percent inter-study scatter of 2D $\Gamma = 2$ Rayleigh–Bénard Nu, and larger than the sub-percent $\text{Pr} = 1 \rightarrow 0.71$ difference [12]. The driver’s internal one-sample standard error (± 0.17) treats a whole decorrelation window as a single sample and overstates the sampling uncertainty by $\sim 7\times$; the block estimate above is the correct one.

Insensitivity to warm-up and window. The time-mean is robust to both analysis choices. Varying the discarded warm-up from 0.05 to 0.30 diffusion times, five settings spanning 133 to 799 turnovers at the fixed full window, moves $\langle \text{Nu} \rangle$ by < 0.002 . Lengthening the averaging window over four settings from 1199 to 3597 turnovers, at a fixed 0.15-diffusion-time warm-up, monotonically tightens the interval ($\pm 0.043 \rightarrow \pm 0.025$) while the mean stays within 0.007, the signature of a converged stationary statistic, not a drifting one.

Grid convergence and cross-code agreement. The temporal study above fixes the mesh at h_{128} . We now refine it. We ran the same $\Gamma = 2$, $\text{Ra} = 10^7$ cell at four resolutions with both GPU solvers, under the short protocol (warm-up 0.05τ , averaging window 0.25τ). The warm-up and window sensitivity above shows that this short window returns the same $\langle \text{Nu} \rangle$ as the 1.5τ window for a given trajectory, so a short run is a valid sample of the time average. What the short window does not remove is the scatter between realizations. In this two-dimensional $\Gamma = 2$ cell the large-scale circulation settles into long-lived states, so different initial conditions can average to values about one per cent apart, and a longer window does not average across states. We therefore report the mean



Supplementary Figure S9. Statistical convergence of the turbulent Rayleigh–Bénard $\langle \text{Nu} \rangle$ ($\Gamma = 2$, $\text{Ra} = 10^7$, $\text{Pr} = 0.71$, h_{128} , one H200). **(a)** Instantaneous plate-averaged $\text{Nu}(t)$ over $\approx 4.0 \times 10^3$ turnover times (grey), the developed-window running mean (blue), the block-averaged $\langle \text{Nu} \rangle = 14.04$ with its 95% interval (red), and the Johnston–Doering 2D-DNS value 13.64 (green dashed); the dotted line marks the warm-up cut. **(b)** Nusselt autocorrelation: fast decorrelation at $\tau_{\text{dec}} \approx 0.9$ turnovers (dashed) with a coherent large-scale-circulation oscillation (≈ 4.5 -turnover period), giving $\approx 4 \times 10^3$ effectively independent samples. **(c)** Flyvbjerg–Petersen blocking curve: we quote the largest 95% interval over the statistically reliable block lengths (≥ 50 blocks), ± 0.025 , attained just above the correlation time; coarser blockings give smaller intervals, so the quoted value is a conservative bound (few-block tail marked with crosses is statistically unreliable).

over independent seeds, with a 95% interval taken across seeds, six seeds per cell except the two most expensive. The single long realization of the temporal study, 14.04, is one such sample. It sits 1.6σ below the six-seed h_{128} ensemble mean 14.15 ± 0.07 , a low draw from the same distribution. The between-seed scatter, not the temporal interval, is the dominant uncertainty.

Supplementary Table S29 gives the mesh sequence, and the two discretizations approach a common value from opposite sides. The colocated CNN rises monotonically from below, with shrinking increments, and crosses the Johnston–Doering correlation between h_{128} and h_{256} . The unstructured P_2 – P_1 GNN falls sharply from h_{64} to h_{128} and then flattens near 14.15. At the finest mesh each solver reaches, the two agree to about one per cent. The CNN at h_{512} gives 14.01 ± 0.20 and the GNN at h_{256} gives 14.16 ± 0.18 , and the window-converged GNN h_{128} realization, 14.04, matches the CNN h_{512} value to 0.2%. Both land near 14.0, about 2.7 to 3.8% above the correlation.

Two independent discretizations approaching the same time-averaged Nusselt number, within their scatter, is mutual corroboration of the two discretizations at their respective finest meshes. Neither sequence is Richardson-extrapolated and the CNN is still rising at h_{512} , so this is corroboration rather than a converged value. We read the residual above the Johnston–Doering value as a real discretization-versus-reference effect, not as convergence to the correlation itself. It is comparable to the few-per-cent inter-study

scatter reported for two-dimensional $\Gamma = 2$ Rayleigh–Bénard [12] and larger than the sub-percent $\text{Pr} = 1 \rightarrow 0.71$ difference. We did not run the GNN at h_{512} , where a single 1.5τ trajectory would cost about 160 wall-clock hours, and the GNN h_{256} interval is wider because it rests on four seeds rather than six.

Supplementary Table S29. Mesh dependence of the turbulent-RBC $\langle \text{Nu} \rangle$ ($\Gamma = 2$, $\text{Ra} = 10^7$, $\text{Pr} = 0.71$, short protocol). Seed-ensemble mean and 95% interval across the n seeds of each entry. The two solvers approach a common value near 14.0 from opposite sides: the CNN rises from below, the GNN falls from above. $\Delta\%$ is relative to the Johnston–Doering correlation $0.138 \text{Ra}^{0.285} = 13.64$. The GNN was not run at h_{512} (a single 1.5τ trajectory is ≈ 160 wall-clock hours).

Mesh	CNN (structured FD)			GNN (P_2 – P_1)		
	$\langle \text{Nu} \rangle$	$\Delta\%$	n	$\langle \text{Nu} \rangle$	$\Delta\%$	n
h_{64}	13.22 ± 0.09	-3.1	6	14.65 ± 0.02	$+7.4$	6
h_{128}	13.52 ± 0.01	-0.9	6	14.15 ± 0.07	$+3.7$	6
h_{256}	13.80 ± 0.24	$+1.2$	6	14.16 ± 0.18	$+3.8$	4
h_{512}	14.01 ± 0.20	$+2.7$	6			

Wall-clock time to each solver’s own converged statistic. On one H200, reaching each solver’s own stationary $\langle \text{Nu} \rangle$ takes 1.15 h for the CNN against 1.15 h for the GNN at h_{64} , and 2.13 h against 5.55 h at h_{128} . These walls are reconstructed as steps-to-convergence $\times$ per-step cost (Supplementary Table S30; Section 4.2), and the statistic is time-averaged over 0.25 thermal-diffusion times (≈ 666 turnovers) past a 0.05-diffusion-time warm-up (three-seed mean). The two solvers converge to $\langle \text{Nu} \rangle$ values that differ by up to 10% at a given mesh, so these are costs to each solver’s own stationary statistic, not to a shared target. The meshes are matched by resolution, not by DOF: the colocated CNN carries $\approx 3.3\times$ fewer unknowns than the P_2 – P_1 GNN.

Supplementary Table S30. Per-step wall time on turbulent Rayleigh–Bénard ($\text{Ra} = 10^7$, $\Gamma = 2$; main-text Figure 3d,e). CNN = structured finite difference (GPU); GNN = unstructured P_2 – P_1 RK3–CN (GPU); CPU FEM = the *identical* scheme and discretization at the same tolerances (10^{-6} pressure, 10^{-10} mass/diffusion), on 32 cores with PETSc CG + GAMG and not thread-tuned. Per-step is the developed-cost least-squares slope at matched time step; the last column is the CPU/GPU ratio of that identical step (Tier B, Section 4.2).

Mesh	CNN (ms)	GNN (ms)	CPU FEM (ms)	CPU/GPU (FEM)
h_{64}	6.6	76.8	167.3	2.2
h_{128}	7.5	171.2	613.8	3.6

S11 Numerical methods and reproducibility

This note documents the discrete equations, the exact solver configuration, and the hardware and software stack, so that the benchmark can be reimplemented independently. All runs use double precision (fp64) throughout.

S11.1 Steady SIMPLE equations (cavity)

On the P_2 – P_1 Taylor–Hood pair the steady cavity is advanced by a pseudo-transient SIMPLE iteration. Each outer iteration solves the relaxed momentum system for a provisional velocity $\mathbf{u}^*$,

$$\left[\mathbf{A}(\mathbf{u}) + \frac{1-\alpha_U}{\alpha_U} \text{diag}(a_P) \right] \mathbf{u}^* = \frac{1}{\Delta t_\tau} \mathbf{M} \mathbf{u}^n + \frac{1-\alpha_U}{\alpha_U} \text{diag}(a_P) \mathbf{u}^n + \mathbf{f} - \mathbf{G} p, \quad (\text{S5})$$

where $\mathbf{A}(\mathbf{u}) = \frac{1}{\Delta t_\tau} \mathbf{M} + \text{Pr} \mathbf{K} + \mathbf{N}(\mathbf{u})$ and $a_P \equiv \text{diag} \mathbf{A}$ is the *unrelaxed* diagonal, taken after any deferred-correction diffusion has been added. Under-relaxation is applied implicitly, by adding $\frac{1-\alpha_U}{\alpha_U} a_P$ to the diagonal and the matching term to the right-hand side, so the converged fixed point is independent of α_U . Here $\mathbf{M}, \mathbf{K}$ are the mass and stiffness operators, $\mathbf{N}(\mathbf{u})$ the convective (non-conservative Galerkin) advection operator, $\mathbf{f}$ the buoyancy load $+\text{Ra} \text{Pr} \mathbf{M} T$ on the vertical component, and Δt_τ the pseudo-time step. The discrete gradient is $\mathbf{G}_{d,ij} = \int \phi_i \partial_d \psi_j$ with ϕ the P_2 velocity and ψ the P_1 pressure basis, so $\mathbf{G}^\top \mathbf{u} = -\int \psi (\nabla \cdot \mathbf{u})$: it is the negative of the weak divergence, and the signs below follow that convention. The pressure correction p' solves the Schur-complement (pressure-Poisson) system and corrects the velocity,

$$\mathbf{S} p' = \mathbf{G}^\top \mathbf{u}^*, \quad \mathbf{S} = \mathbf{G}^\top \text{diag}(r_{AU}) \mathbf{G}, \quad r_{AU} = \frac{1}{a_P}, \quad \mathbf{u} = \mathbf{u}^* - r_{AU} \mathbf{G} p', \quad p \leftarrow p + \alpha_p p', \quad (\text{S6})$$

with under-relaxation $(\alpha_U, \alpha_p, \alpha_T)$. Note that r_{AU} uses the unrelaxed a_P ; carrying α_U into r_{AU} instead would rescale p' by $1/\alpha_U$ and, at the high-Ra setting $\alpha_U = \alpha_p$, remove the pressure under-relaxation entirely. The pressure gauge is fixed by subtracting the arithmetic mean from the right-hand side and pinning one P_1 degree of freedom; the graph-multigrid operator is anchored on that row. The matrix-free operator $\mathbf{S}$ is applied as $\mathbf{S} v = \sum_d \mathbf{G}_d^\top (r_{AU} \odot \mathbf{G}_d v)$ (never assembled). The unsteady Rayleigh–Bénard and conjugate cases instead use the three-stage rotational IPCS RK3–CN scheme of Section 4.3; the pressure Poisson solve is the same graph multigrid.

S11.2 Solver configuration

Supplementary Table S31 lists the exact settings. The smoothed-aggregation hierarchy is supplied the constant near-null-space vector for the Neumann operator. No SUPG/PSPG or grad–div stabilization is used. Where deferred correction is enabled (Supplementary

Table S31) an artificial-diffusion operator $\gamma_{\text{mom}}\mathbf{K}$ is added implicitly with the matching explicit term, $\gamma_{\text{mom}} = \max(0, \text{Pe} - 1)\text{Pr}$ and $\text{Pe} = |\mathbf{u}|_{\text{max}} h_{\text{min}}/(2\text{Pr})$; it vanishes at the fixed point but affects the convergence rate and hence the reported wall-clock times. Pressure stability comes from the inf-sup-stable Taylor–Hood pair, and advection is stabilized only by the pseudo-transient term and deferred correction.

Supplementary Table S31. Exact solver configuration (production runs; fp64 throughout). All four solvers run steady SIMPLE on the cavity. On Rayleigh–Bénard and the conjugate cavity the GNN and FEniCSx implementations use the transient rotational-IPCS RK3–CN march; on Rayleigh–Bénard the CNN uses its own explicit Heun predictor–corrector with a convolutional-multigrid projection (Section 4.3). That is the structured solver as its construction defines it, and it is why that comparison is reported as cross-method rather than same-method.

Item	Value
Element pair	P_2 – P_1 Taylor–Hood (velocity/temperature P_2 , pressure P_1)
Advection form	convective (non-conservative) Galerkin $N_i \mathbf{u} \cdot \nabla N_j$, deferred correction
Quadrature	degree 5 (barycentric); no dealiasing/over-integration
Stabilization	none (inf-sup-stable pair); deferred-correction diffusion $\gamma_{\text{mom}}\mathbf{K}$ on the 2D and 3D cavity Taylor–Hood runs, vanishing at the fixed point
Mass matrix	consistent (Jacobi-PCG). Relative tolerance 10^{-10} on the single-GPU conjugate cavity, in both implementations, and on both implementations of the Rayleigh–Bénard per-step pair (Supplementary Table S30); 10^{-7} on the single-GPU Rayleigh–Bénard production runs ($\langle \text{Nu} \rangle$ statistics) and on the distributed Rayleigh–Bénard and conjugate runs. Rotational term uses P_1 lumped mass
SIMPLE under-relaxation	$\alpha_U/\alpha_p/\alpha_T = 0.7/0.3/0.9$ ($\alpha_U \rightarrow 0.5$ at $\text{Ra} \geq 10^5$, 0.3 at $\text{Ra} \geq 10^6$)
Pseudo-time step	$\Delta t_\tau = 10^5 h^2/\text{Pr}$, clamped $\leq 1/\text{Pr}$ ($\text{Ra} \geq 10^6$), $\leq 3/\text{Pr}$ ($\text{Ra} \geq 10^5$)
Cavity inner tolerances	momentum/energy/pressure rtol 10^{-6} ; pressure max iter 100
Transient scheme	rotational-IPCS, RK3 advection + Crank–Nicolson diffusion
Transient Δt (RBC)	adaptive, CFL target 1.0 (high-water-mark)
Transient Δt (conjugate)	<i>fixed</i> , sized once at start-up from $\text{CFL} \cdot h_{\text{min}}/u_{\text{char}}$; shared by both implementations in Supplementary Table S22
CNN transient Δt (RBC)	explicit, $\Delta t = 0.1 \Delta x/\sqrt{\text{Ra}}\text{Pr}$ (e.g. 2.93×10^{-7} at h_{128} , $\text{Ra} = 10^7$)
Pressure solver	CG preconditioned by one graph-multigrid V-cycle, rtol 10^{-6}
Pressure iteration cap	100 (cavity); 50 (single-GPU RBC, conjugate); 500 for the distributed-cavity scaling sweeps; 300 per solve for the distributed transient runs. The last two are raised so the reported counts are convergence, not truncation. Supplementary Note S3.1 names the runs that reach the cap and how those entries are marked
Multigrid	smoothed-aggregation AMG (PyAMG [15]) on the P_1 Laplacian, up to 10 levels, constant near-null-space
Smoother	Chebyshev degree 2, $n_{\text{pre}} = n_{\text{post}} = 2$; λ_{max} by 10-step power iteration, band $[\lambda_{\text{max}}/6, \lambda_{\text{max}}]$ (weighted-Jacobi option $\omega = 0.7$)
Pressure gauge	pin P_1 DOF 0; SA near-null-space = constant
CUDA-graph capture	the preconditioned-CG loop body (matvec + V-cycle) captured once and replayed; distributed CG stays eager
Partitioning	x -slab (3D cavity, heat sink, engine); y -slab (RBC periodic- x ; conjugate cavity, whose vertical solid band and side walls require each slab to span the full x -extent); halo width 1; NCCL point-to-point 1-ring
Nicolaides deflation	coarse space Z = per-rank indicator (one DOF/rank), $E = Z^T \mathbf{S} Z$; per-rank fine MG 5 iterations
Two-level correction	PyAMG SA on the full P_1 Laplacian, replicated per rank; coarse solve = graph-multigrid, 50 iters rtol 10^{-6}
Solid velocity (CHT)	P_2 velocity pinned to $\mathbf{0}$ on solid nodes (hard Dirichlet)

Nusselt-number evaluation. All solvers evaluate Nu from the wall temperature gradient (not a volume average). The GNN and FEniCSx use the *same* P_2 -consistent wall-flux operator of Equation (7) in Methods; the CNN uses the one-sided finite difference wall gradient; OpenFOAM uses its `wallHeatFlux` function object. On the unit cavity, where the wall has unit length and $\Delta T = 1$, these coincide with the conduction-normalized form. For Rayleigh–Bénard $Nu = \frac{1}{2}(Nu_{\text{bot}} + Nu_{\text{top}})$, time-averaged.

S11.3 Hardware and software

Supplementary Table S32 lists the platform. The two GPU solvers (CNN, GNN) and the FEniCSx CPU reference run in separate environments (the CPU finite element reference uses DOLFINx/PETSc); OpenFOAM v12 is a separate build.

Supplementary Table S32. Hardware and software stack. fp64 throughout.

Component	Detail
GPU	NVIDIA H200 SXM, 141 GB HBM3e, NVLink intra-node (single-GPU runs use one device)
CPU	2× AMD EPYC 9454 (48-core), 251 GB RAM
GPU-solver env (CNN, GNN)	Python 3.11, PyTorch 2.2.2+cu118, CUDA 11.8, NCCL 2.19.3
CPU-FEM env (FEniCSx)	Python 3.12, DOLFINx 0.10.0, PETSc 3.25.1, PyAMG 5.3.0, MPICH (mpi4py 4.1.1)
Adjoint-reference env	pyadjoint 2026.4.1 (<code>pyadjoint_ad</code>), dolfinx-adjoint 0.2.1, checkpoint-schedules 1.0.4, on the same DOLFINx 0.10.0 / PETSc 3.25.1 stack. Installed <code>-no-deps</code> into a side directory so the published FEniCSx environment is untouched; two local compatibility patches were needed against DOLFINx 0.10.0 (a <code>dofmaps</code> accessor change, and an exception class widened around the <code>real_element</code> path)
Finite-volume reference	OpenFOAM v12 (double precision), <code>foamRun -solver fluid</code>
Compiler	gcc 14.2.0
Environment	CPU cores: all CPU references (FEniCSx, OpenFOAM) on 32, for the cavity, conjugate and Rayleigh–Bénard cases alike. 32 cores is a comparison setting, not a claimed per-method optimum. GPU runs single-GPU, <code>CUDA_VISIBLE_DEVICES=0</code>
Scheduler	Slurm, one exclusive node per timing run, on one to eight GPUs

S11.4 OpenFOAM stopping and inner-tolerance sensitivity

The steady cavity runs stop on the identical $|\Delta Nu / Nu| < 10^{-5}$ three-window test (Methods); for OpenFOAM the test is applied to its wall-heat-flux trace by an external monitor that terminates the run, so its native `fvSolution` tolerances govern only the inner linear solves (GAMG for p_{rgh} and PBiCGStab for momentum/energy, tolerance 10^{-7} in production). Supplementary Table S33 sweeps that inner tolerance by an order of magnitude in each direction on representative cavity cells. The outer iteration count at which the shared stopping criterion fires is *identical* at every tolerance, so the inner setting does not determine the stopping point; the converged Nu moves by $\lesssim 9 \times 10^{-5}$ relative except at

h_{256} , $Ra = 10^4$ (see text), and tightening to 10^{-8} costs at most 22% wall. The one visible effect is at the slow-converging h_{256} , $Ra = 10^4$ cell (cf. the stopping-tolerance discussion in the caption of Extended Data Table 1): there the loose 10^{-6} setting sits 0.12% low, while the production 10^{-7} and the tightened 10^{-8} agree to all printed digits at 2.2449, the value the GNN reaches under the tightened criterion (the FEniCSx P_2 – P_1 stops lower, at 2.237; Extended Data Table 1 and Supplementary Note S11.5).

Supplementary Table S33. OpenFOAM converged Nu and cost versus inner linear-solve tolerance. Differentially-heated cavity, one repetition per cell (the runs are deterministic to the printed digits); the outer stop is the shared $|\Delta Nu/Nu| < 10^{-5}$ criterion in every case, and the outer iteration count at which it fires is identical across tolerances (fourth column). 10^{-7} is the production setting used throughout the paper.

Mesh	Ra	Nu (10^{-6})	Nu (10^{-7})	Nu (10^{-8})	outer iters	wall $10^{-8}/10^{-6}$
h_{64}	10^4	2.2493	2.2494	2.2495	710	1.14
h_{64}	10^6	9.0546	9.0553	9.0554	710	1.09
h_{256}	10^4	2.2422	2.2449	2.2449	4760	1.22
h_{256}	10^6	8.8388	8.8392	8.8393	4760	1.16

S11.5 Stopping criterion: robustness to a combined test

The steady cavity runs stop on a rate-of-change criterion on the Nusselt number, $|\Delta Nu/Nu| < 10^{-5}$ over three consecutive windows. To check that this reflects convergence of the discrete equations, not merely a stable output functional, we re-ran representative cells with a *combined* criterion that additionally requires a tighter Nusselt tolerance (10^{-6}), a velocity update-norm below 5×10^{-3} , and a divergence below 10^{-3} . Supplementary Table S34 lists the result for both P_2 – P_1 solvers. The combined criterion changes the converged $\langle Nu \rangle$ by at most 0.05% for the GNN and not at all, to five figures, for FEniCSx, so for most cells the reported values are converged discrete solutions whose ranking does not depend on the stopping test.

The exception is h_{256} , $Ra = 10^4$, where the GNN and FEniCSx differ by 0.35% under the production criterion (2.2445 against 2.2367), and by a similar margin under the combined criterion (2.2449 against 2.2366). We report this cell as an unresolved outlier. What is established: the two P_2 – P_1 codes assemble bit-identical mass, stiffness, gradient and pressure operators from the same UFL forms (a direct comparison gives identical FEM matrices, and the custom GPU advection matrix reproduces the compiled finite element form to a relative $\sim 10^{-14}$) and evaluate the wall-Nusselt number with the same functional, so the difference is not one of discretization. The GNN value matches the independent cell-centred OpenFOAM finite volume result (2.24488) and the de Vahl Davis benchmark (2.243), and the gap shrinks to $\approx 0.05\%$ at h_{128} . The GNN reaches 2.2449 only after roughly three times as many SIMPLE iterations as the FEniCSx run

takes to stop. The suspected cause is therefore incomplete convergence of the FEniCSx run: its Nusselt-rate and update-norm criteria are met on a slow plateau at the finest, slowest-converging mesh. We have not confirmed this, because the direct test, running the FEniCSx h_{256} cell to the GNN’s iteration count (≈ 40 CPU-hours), was not affordable. This is why we cross-check against the independent references and drop this cell from the refinement trend.

Supplementary Table S34. Stopping-criterion robustness. Converged $\langle \text{Nu} \rangle$ under the Nusselt-only criterion and under a combined criterion (Nu tolerance 10^{-6} , velocity update norm $< 5 \times 10^{-3}$, divergence $< 10^{-3}$), for the two P₂–P₁ solvers on representative cavity cells. Δ is the change from the Nusselt-only value; r_u is the velocity update norm at the Nusselt-only stop (≈ 0 = below printed precision).

Cell	Solver	$\langle \text{Nu} \rangle$ (Nu-only)	r_u at stop	$\langle \text{Nu} \rangle$ (combined)	Δ (%)
h_{128} , Ra= 10^5	GNN	4.5218	2.7×10^{-3}	4.5220	0.004
h_{128} , Ra= 10^5	FEniCSx	4.5221	≈ 0	4.5221	0.000
h_{256} , Ra= 10^4	GNN	2.2445	2.0×10^{-4}	2.2449	0.018
h_{256} , Ra= 10^4	FEniCSx	2.2367	≈ 0	2.2367	0.000
h_{256} , Ra= 10^5	GNN	4.5187	7.2×10^{-3}	4.5211	0.053

S11.6 Data and code availability

The deposit released on publication, in a public repository with a citable DOI, contains four items: the minimal working example described in Code availability; the production Hydra configuration groups (conf/: solver, problem, mesh, ra, relaxation, convergence, pc, partition, mg/smooth); the voxelized and body-fitted meshes (.npz); and the per-run result logs (JSON, each carrying its resolved configuration and a configuration hash). The production solver source is not part of it. The example accompanies this submission for the editor and reviewers under review terms, with no licence granted at this stage. It carries two configuration groups of its own and no meshes, so the groups listed above are the production solver’s. The reference Nusselt values are de Vahl Davis [13], Le Quéré [14], Tric [9], and Johnston–Doering [11].

Supplementary References

- [1] Christophe Geuzaine and Jean-François Remacle. Gmsh: A 3-D finite element mesh generator with built-in pre- and post-processing facilities. *International Journal for Numerical Methods in Engineering*, 79(11):1309–1331, 2009. doi: 10.1002/nme.2579.
- [2] Boyang Chen, Claire E. Heaney, and Christopher C. Pain. Neural Physics: Using AI libraries to develop physics-based solvers for incompressible computational fluid

dynamics. *Computers & Fluids*, 308:106981, 2026. doi:
 10.1016/j.compfluid.2026.106981. Preprint: <https://arxiv.org/abs/2402.17913>.
 Software: <https://github.com/ImperialCollegeLondon/AI4PDEs>.

- [3] Igor A. Baratta, Joseph P. Dean, Jørgen S. Dokken, Michal Habera, Jack S. Hale,
 Chris N. Richardson, Marie E. Rognes, Matthew W. Scroggs, Nathan Sime, and
 Garth N. Wells. DOLFINx: The next generation FEniCS problem solving
 environment. Preprint, Zenodo, 2023. doi: 10.5281/zenodo.10447666.
- [4] Sebastian K. Mitusch, Simon W. Funke, and Jørgen S. Dokken. dolfin-adjoint
 2018.1: automated adjoints for FEniCS and Firedrake. *Journal of Open Source
 Software*, 4(38):1292, 2019. doi: 10.21105/joss.01292.
- [5] M. Khatamifar, W. Lin, and L. Dong. Transient conjugate natural convection heat
 transfer in a differentially-heated square cavity with a partition of finite thickness
 and thermal conductivity. *Case Studies in Thermal Engineering*, 25:100952, 2021.
 doi: 10.1016/j.csite.2021.100952.
- [6] J. M. House, C. Beckermann, and T. F. Smith. Effect of a centered conducting
 body on natural convection heat transfer in an enclosure. *Numerical Heat Transfer,
 Part A: Applications*, 18(2):213–225, 1990. doi: 10.1080/10407789008944791.
- [7] ANSYS, Inc. Conjugate heat transfer simulation of a single-cylinder internal
 combustion engine. In *Ansys Forte Tutorials*, Release 2025 R1, chapter 24, 2025.
 Accessed via the ANSYS Help system, 26 August 2026.
- [8] J. B. Heywood. *Internal Combustion Engine Fundamentals*. McGraw-Hill
 Education, New York, 2nd edition, 2018. Ch. 12, heat transfer in engines.
- [9] E. Tric, G. Labrosse, and M. Betrouni. A first incursion into the 3D structure of
 natural convection of air in a differentially heated cubic cavity, from accurate
 numerical solutions. *International Journal of Heat and Mass Transfer*, 43(21):
 4043–4056, 2000. doi: 10.1016/S0017-9310(00)00037-5.
- [10] H. Flyvbjerg and H. G. Petersen. Error estimates on averages of correlated data.
The Journal of Chemical Physics, 91(1):461–466, 1989. doi: 10.1063/1.457480.
- [11] Hans Johnston and Charles R. Doering. Comparison of turbulent thermal
 convection between conditions of constant temperature and constant flux. *Physical
 Review Letters*, 102(6):064501, 2009. doi: 10.1103/PhysRevLett.102.064501.
- [12] Erwin P. van der Poel, Richard J. A. M. Stevens, and Detlef Lohse. Comparison
 between two- and three-dimensional Rayleigh–Bénard convection. *Journal of Fluid
 Mechanics*, 736:177–194, 2013. doi: 10.1017/jfm.2013.488.

- 2155 [13] G. de Vahl Davis. Natural convection of air in a square cavity: A bench mark
2156 numerical solution. *International Journal for Numerical Methods in Fluids*, 3(3):
2157 249–264, 1983. doi: 10.1002/fld.1650030305.
- 2158 [14] P. Le Quéré. Accurate solutions to the square thermally driven cavity at high
2159 Rayleigh number. *Computers & Fluids*, 20(1):29–41, 1991. doi:
2160 10.1016/0045-7930(91)90025-D.
- 2161 [15] Nathan Bell, Luke N. Olson, Jacob Schroder, and Ben Southworth. PyAMG:
2162 Algebraic multigrid solvers in Python. *Journal of Open Source Software*, 8(87):
2163 5495, 2023. doi: 10.21105/joss.05495.